

La Grammaire de l'Exécution : Un Catalogue Universel des Commandes Informatiques de l'Industrie à la Synthèse Biologique

Fondements du Concept de Commande : Au-delà de l'Interface Standard

L'établissement d'un répertoire exhaustif des commandes informatiques exige une définition rigoureuse et étendue du terme « commande ». L'objectif de recherche, tel qu'exprimé, ne se limite pas aux entrées acceptées par un shell Unix traditionnel ou une invite de commandes Windows ; il englobe toute forme d'information structurée susceptible d'être interprétée comme une instruction valide par un système informatisé quelconque . Cette perspective élargie doit prendre en compte les critères de validité tels que l'exécution, le test, la reproductibilité, la falsifiabilité, ainsi que des entrées plus subtiles comme les logs, les constantes opérationnelles, les mesures ou les ajustements . Par conséquent, une commande est définie comme toute séquence d'information, quelle que soit sa forme syntaxique, qui peut provoquer un changement d'état, déclencher une action spécifique, initier un calcul ou générer une sortie mesurable dans un système donné. Cette approche transcende la simple distinction entre code source et données pour embrasser une vision unifiée de l'instruction.

La première catégorie de commandes concerne les systèmes d'exploitation et les environnements d'exécution standard. Ces commandes sont directement interprétées par un shell, un programme qui agit comme un interpréteur de commandes pour l'utilisateur ²⁸⁵³⁸⁷. Les shells comme Bash, zsh, Fish sur les systèmes Unix-like (Linux, macOS) ou PowerShell sous Windows représentent la norme pour l'interaction avec les systèmes d'exploitation ^{22 65 240}. Ils permettent d'exécuter des programmes binaires, de manipuler des fichiers et des processus, et de construire des scripts automatisant des tâches complexes ³⁹ . La norme POSIX (Portable Operating System Interface) fournit un cadre pour la portabilité de ces commandes entre différentes implémentations de shells, garantissant un comportement cohérent ¹⁰⁴¹⁷⁵. Au-delà des shells interactifs, les

interfaces de ligne de commande (CLI) sont omniprésentes dans les outils modernes, allant des compilateurs aux gestionnaires de paquets comme Nix, en passant par les outils de gestion d'infrastructure comme Terraform et Pulumi ⁷⁸. Chaque invocation de `terraform apply` ou `pulumi up` est une commande complexe qui orchestre des actions sur des infrastructures distantes.

Les langages de script et de haut niveau complètent cette couche en offrant des moyens puissants d'automatisation et de développement généraliste. Des langages comme Python, Ruby, Perl, et PHP sont conçus pour la rapidité de développement et l'automatisation de tâches diverses, depuis la manipulation de fichiers jusqu'à l'interaction avec des API web ³⁹¹. Python, en particulier, a acquis une position centrale grâce à sa versatilité, son vaste écosystème de bibliothèques et son adoption massive dans le domaine de l'intelligence artificielle et de l'analyse de données ³⁹⁶. De même, des langages fonctionnels et logiques comme Haskell, Prolog, Erlang et F# offrent des paradigmes de programmation distincts, privilégiant la pureté fonctionnelle, la concurrence massive ou la programmation déclarative pour modéliser des problèmes complexes. Ces langages, souvent exécutés via un interpréteur ou un moteur virtuel, constituent une autre grande famille de commandes exécutables.

Au cœur du système, les langages de bas niveau et les langages système fournissent le lien direct avec le matériel. Le langage C reste un pilier fondamental pour le développement d'OS, de pilotes et d'applications nécessitant un contrôle fin sur la mémoire et les ressources processeur. Son descendant direct, C++, étend ce pouvoir avec des paradigmes orientés objet. Des langages plus récents comme Rust, Zig et V (Vlang) visent à combiner la performance de C avec des garanties de sécurité accrues (par exemple, la gestion mémoire sans ramasse-miettes dans Rust) ou une meilleure pragmatique et lisibilité. Le langage Assembleur (ASM) représente l'abstraction ultime, où chaque instruction correspond directement à une opération de la machine, rendant possible le pilotage de l'unité centrale de traitement (CPU) au plus près du silicium. Ensemble, ces langages forment la base de tout système informatique, transformant des intentions abstraites en micro-ordres électriques.

Enfin, une dimension essentielle de la commande réside dans les langages de balisage et de format de données. Bien que principalement conçus pour représenter et échanger des informations, ils peuvent également encapsuler des instructions d'exécution. XML et JSON sont des formats ubiquitaires pour la sérialisation de données dans les applications web et distribuées. Dans certains contextes, comme dans les appels de procédure distante (RPC) sur JSON-RPC, des structures JSON peuvent contenir des noms de méthodes et des paramètres, agissant ainsi comme des commandes exécutables par un serveur distant ⁷⁸. De même, des formats comme YAML sont largement utilisés pour la

configuration d'applications et de pipelines CI/CD, leurs directives étant interprétées par les outils pour configurer leur propre comportement . Les formats de sérialisation performants comme Protocol Buffers, Cap'n Proto ou FlatBuffers, développés par Google, sont conçus pour la communication entre services distribués et peuvent transporter des commandes structurées avec une efficacité maximale ⁸³ . La frontière entre un simple format de données et un langage de commandes devient floue lorsque ces formats sont intégrés dans des architectures de service qui les interprètent comme des invocations d'API. Cette vision holistique est cruciale pour construire un répertoire exhaustif qui ne se contente pas de lister des langages, mais qui comprend la multitude de façons dont l'information peut être structurée pour donner des ordres à une machine.

Catégorie de Commande	Exemples de Langages/Formats	Dispositifs Cibles	Intention Principale
Interfaces de Commande (CLI)	Bash, PowerShell, Zsh, Fish ²⁴⁰	OS (Linux, Windows, macOS), Outils DevOps ¹³⁹	Automatisation, Gestion système, Interaction utilisateur ²¹⁰
Langages de Scripting	Python, Lua, Perl, Ruby	Serveurs, Applications web, Systèmes embarqués ⁶¹	Automatisation rapide, Colle logicielle, Développement web ³⁹²
Langages Système & Bas-Niveau	C, C++, Rust, Zig, Assembly	OS, Pilotes, Logiciels embarqués critiques ³⁴⁵	Contrôle matériel, Performance, Sécurité mémoire ³⁴⁶
Langages Fonctionnels/Logiques	Haskell, Prolog, Erlang, F#	IA, Systèmes distribués, Vérification formelle ⁶⁷	Programmation déclarative, Concurrence, Raisonnement ⁶⁶
Langages de Balisage/Sérialisation	HTML, CSS, XML, JSON, YAML	Navigateurs web, API REST, Configuration d'infrastructures ²⁷⁵	Structure de données, Représentation visuelle, Communication ⁷⁸

Cette première approche établit une base solide, identifiant les piliers de l'informatique moderne. Cependant, pour atteindre l'objectif d'une exhaustivité absolue, il est impératif de plonger dans les domaines spécialisés où les concepts de commande et d'interface prennent des formes très variées et souvent inattendues, allant de la fabrication assistée par ordinateur à la téléphonie mobile en passant par la robotique autonome.

Le Monde Industriel : Protocoles et Langages de Contrôle Automatisé

Le secteur de l'automatisation industrielle constitue un laboratoire riche et spécialisé pour l'étude des commandes informatiques, loin des environnements de bureau standards. Il est dominé par des dispositifs conçus pour interagir de manière fiable et en temps réel avec le monde physique, ce qui a conduit au développement de protocoles et de langages

de programmation hautement optimisés pour la supervision et le contrôle des processus manufacturiers, énergétiques et de production. Les trois piliers de cet univers sont les Automates Programmables Logiques (APL), les Interfaces Homme-Machine (IHM) et les instruments de mesure scientifique, chacun ayant ses propres modes de commande.

Les Automates Programmables Logiques (PLC) sont les cerveaux des systèmes de contrôle industriel. Pour assurer l'interopérabilité entre équipements de fabricants différents, l'IEC 61131-3 est devenu la norme internationale qui définit un ensemble de cinq langages de programmation standardisés ²⁶⁷³⁰³. Ces langages, disponibles dans les environnements de développement intégrés (EDIs) comme Siemens TIA Portal ou Rockwell Studio 5000, permettent aux ingénieurs de programmer des automatismes complexes ¹¹⁰. Ils se divisent en deux catégories principales : graphiques et textuels. Les langages graphiques, comme le Schéma à Relais (LD) et le Schéma à Blocs Fonctionnels (FBD), offrent une visualisation intuitive des circuits logiques et des flux de données, respectivement, et sont très populaires auprès des ingénieurs en automatisme habitués au dessin de schémas électriques ^{42 388}. Les langages textuels, Structured Text (ST) et Instruction List (IL), offrent une flexibilité et une puissance supérieures pour des algorithmes complexes. ST, similaire à un langage de type Pascal ou C, permet de coder des logiques de contrôle sophistiquées, des boucles, des fonctions et des structures de données, tandis que IL est une représentation textuelle proche de l'assembleur, offrant un contrôle fin sur les opérations du PLC ¹⁶⁷¹⁶⁸¹⁷². Une cinquième langue, Sequential Function Chart (SFC), est utilisée pour modéliser les processus comme une série d'étapes et de transitions, facilitant la conception de machines séquentielles ⁴⁷. Les commandes dans ce contexte sont donc des blocs de code écrits dans l'un de ces langages, traduits en micro-instructions exécutées par le processeur du PLC pour lire des capteurs (entrées) et commander des actionneurs (sorties).

Les Interfaces Homme-Machine (IHM), souvent appelées panneaux tactiles, sont la face visible de l'automatisation. Elles permettent aux opérateurs humains d'interagir avec les PLC, d'afficher les états des processus, de consulter des tendances historiques et de contrôler manuellement les équipements ³⁵⁵. La communication entre l'HMI et le PLC est presque toujours assurée par le protocole Modbus, un standard ouvert et extrêmement répandu en industrie ^{9 182}. Modbus définit une structure de message simple qui permet à un maître (l'HMI ou un PC supervising) de lire ou d'écrire des données dans des registres mémoire virtuels du PLC slave (l'automate) ²⁵¹. Ces registres correspondent à des valeurs analogiques (vannes, températures), des bits logiques (moteurs, lampes) ou des mots de données ¹⁵¹. Une commande HMI consiste donc à envoyer une trame Modbus spécifique : par exemple, une requête de lecture de registre pour afficher une valeur ou une requête d'écriture de bit pour démarrer un moteur. Cette interaction peut

se faire sur des bus série (RS-485 en mode RTU ou ASCII) ou sur Ethernet (Modbus TCP) 145146. Des logiciels HMI open-source comme AdvancedHMI ou des solutions commerciales comme Indusoft permettent de créer ces interfaces graphiques et de les connecter facilement aux PLC via Modbus 3 204. L'utilisation de commandes CLI comme `modbus-cli` permet même de tester et de diagnostiquer ces communications directement depuis une station de travail, montrant que les outils d'administration réseau sont parfaitement adaptés à l'industrie 132136.

Le troisième pilier de l'automatisation industrielle est celui des instruments de mesure et de test. Pour automatiser les laboratoires de R&D, de production et de maintenance, le Standard Commands for Programmable Instruments (SCPI) a été développé 85. SCPI est une spécification de commandes ASCII hiérarchiques et arborescentes, normalisée par IEEE 488.2, qui définit un ensemble commun de commandes pour tous les types d'instruments (oscilloscopes, analyseurs de spectre, générateurs de signaux, alimentations) 87 89. Une commande SCPI typique ressemble à `:MEASurement:VOLTage:DC?` pour lire une tension continue ou `:SYSTem:ERRor?` pour obtenir le dernier code d'erreur 4 186. Cette structure garantit que si un ingénieur connaît les commandes SCPI pour un oscilloscope Keysight, elles seront très probablement applicables à un modèle d'un autre fabricant compatible SCPI 101. Ces commandes sont transmises via des bus physiques comme GPIB (IEEE 488.1), USB ou Ethernet 328. L'existence d'outils comme le VeEX SCPI Remote & Command Reference Tool montre comment ces commandes sont non seulement utilisées dans des scripts d'automatisation, mais aussi apprises et testées de manière interactive 4. En somme, dans l'industrie, la commande est omniprésente, qu'elle soit codée dans un langage de programmation PLCTM, encapsulée dans une trame Modbus, ou formulée comme une chaîne de caractères ASCII conforme à la norme SCPI.

Domaine Industriel	Technologie Clé	Langage/Protocole	Type de Commande	Exemple de Commande
Automatisation (PLC)	Automate Programmable Logique (APL)	IEC 61131-3 (LD, FBD, ST, IL, SFC) ⁴²	Blocs d'instructions logiques et de contrôle ⁴⁷	IF MotorRunning AND Temp > 100 THEN Alarm := TRUE; END_IF;
Supervision (HMI)	Interface Homme-Machine (IHM)	Modbus (RTU/TCP) ⁹	Lecture/Ecriture sur registres (coils, inputs, holding registers) ²⁴⁹	Read Coils (SlaveID=1, StartAddr=0, Num=1)
Mesure (Instruments)	Oscilloscope, Analyseur de Spectre	SCPI (Standard Commands for Programmable Instruments) ⁸⁵	Chaînes de caractères hiérarchiques pour configuration et acquisition ⁸⁷	:MEASure:VOLTage:DC? (@1)
Fabrication CNC	Machine-Outil à Commande Numérique	G-code	Instructions de mouvement et de fonctionnement de la machine	G0 X10 Y15 (déplacement rapide), M03 S1500 (marche broche)

Ce panorama montre que le concept de "commande" dans l'industrie est profondément ancré dans des standards ouverts et des langages spécialisés conçus pour la robustesse, la fiabilité et l'interopérabilité. Ces systèmes, bien que distincts des ordinateurs personnels, constituent une partie intégrante et massivement utilisée de l'écosystème informatique mondial, et leur inclusion dans un répertoire exhaustif est donc indispensable.

Télécommunications et Interfaces Physiques : Commandes sur les Réseaux Mobiles et Téléphoniques

Le domaine des télécommunications et des interfaces physiques offre une perspective fascinante sur les commandes, car il illustre comment des signaux physiques simples peuvent être codifiés pour transmettre des instructions complexes. Contrairement aux environnements informatiques où l'entrée se fait principalement via un clavier, ici, l'interface homme-machine est souvent une surface tactile, un écran, un microphone ou, historiquement, un clavier à touches numériques. Les dispositifs concernés sont omniprésents : téléphones fixes, smartphones (Samsung, iPhone), PBX (Private Branch Exchange) et autres équipements de téléphonie IP ³⁵ ⁶³.

Un des exemples les plus anciens et les plus répandus de commande physique est le signal DTMF (Dual-Tone Multi-Frequency). Lorsqu'un utilisateur compose un numéro de téléphone ou navigue dans un menu vocal interactif (IVR), il génère une séquence de tonalités DTMF ⁶². Chaque touche du clavier correspond à une paire unique de fréquences basses et hautes, ce qui permet une détection robuste même sur des lignes

téléphoniques analogiques bruitées ¹⁷⁸. Bien que principalement conçu pour le transport de numéros, le DTMF est intrinsèquement un moyen de transmission de données binaire. Dans les systèmes de téléphonie comme Asterisk, une application PBX open-source, le DTMF est utilisé pour exécuter des commandes. Le dialplan d'Asterisk est un ensemble de règles qui décident quoi faire lorsqu'un appel arrive ; il peut être configuré pour attendre une entrée DTMF spécifique et ensuite exécuter une action, comme transférer l'appel, jouer un message audio ou même exécuter un script externe ²²⁰²²³. Des commandes comme `Background()` jouent un son et suspendent l'exécution jusqu'à ce qu'une touche soit pressée, tandis que `WaitForDtmf()` lit une seule touche ²²¹. Ce mécanisme transforme le simple acte de taper sur un téléphone en une interaction avec un système complexe, où chaque tonalité est une pièce du puzzle de la commande. Des modules comme `app_dtmf` permettent de gérer des séquences de touches DTMF pour contrôler des applications personnalisées ²²⁰. L'AT Command Set, un langage de contrôle historique pour les modems, utilise également des commandes pour générer des tonalités DTMF, montrant une convergence entre les interfaces vocales et les commandes de périphériques ²²⁷.

Pour les communications mobiles (GSM, 3G, 4G/LTE, 5G), le protocole USSD (Unstructured Supplementary Service Data) remplit une fonction similaire au DTMF, mais sur un réseau de données. Contrairement au SMS, qui est asynchrone et délivré sans garantie, une session USSD est synchrone et crée une connexion en temps réel entre le téléphone et le serveur du réseau ²²⁶. L'utilisateur saisit une chaîne de caractères spéciale, typiquement commençant par un astérisque (`<em>`) et se terminant par un dièse (`#`), comme `</em>123#` ²²⁵. Cette chaîne est envoyée au serveur USSD du fournisseur, qui la traite et renvoie une réponse, souvent sous forme de texte à afficher à l'écran ou de nouveau menu. Les applications sont nombreuses : consultation du solde, activation de services, enregistrement de points de fidélité, etc. ⁶³. Sur les smartphones, notamment les appareils Android, l'accès à ces services est souvent transparent pour l'utilisateur, mais les applications sous-jacentes interagissent directement avec le module radio pour envoyer des requêtes USSD et traiter les réponses ⁶³. Par exemple, une application bancaire pourrait utiliser une requête USSD pour vérifier le solde d'un compte sur un réseau mobile prépayé. Le standard 3GPP TS 22.090 définit précisément ce protocole, qui est conçu pour être utilisé par des applications du réseau et leurs homologues sur le téléphone (UE - Unité d'Équipement) ²²⁵²²⁸. Le USSD est donc une interface de commande directe et instantanée sur le réseau mobile lui-même.

Dans le contexte des assistants vocaux et des voitures intelligentes, la commande prend la forme de l'interaction vocale. Des systèmes comme ceux de Tesla, Mercedes ou BMW permettent aux conducteurs de contrôler de nombreux aspects de leur véhicule —

navigation, climatisation, musique — uniquement par la voix ^{98 99}. L'interaction avec un Tesla peut être initiée en parlant des commandes comme "Keep my car safe" pour activer le mode Sentry ⁴⁰, ou en demandant "Play playlist 'Road Trip'" ⁴¹. Derrière ces phrases se cache un système de reconnaissance vocale et de traitement du langage naturel (TLN) qui les convertit en commandes structurées. Par exemple, la demande "I'm cold" est interprétée comme une instruction pour augmenter la température du chauffage ⁹⁸. Ces commandes peuvent être envoyées via des API, comme la Tesla Vehicle Command SDK, qui utilise le Tesla Vehicle Command Protocol pour exécuter des actions sur le véhicule ¹⁵⁷²⁷⁰. De même, sur les iPhones, les Shortcuts peuvent être déclenchés via Siri, acceptant des entrées textuelles sélectionnées dans d'autres applications pour les traiter ^{37 38}. Apple a également défini des standards comme Matter, qui permet de contrôler des appareils intelligents via des assistants comme Siri, Alexa ou Google Assistant, unifiant ainsi les commandes vocales à travers différents écosystèmes ²⁷²²⁷⁴. Sur Android Automotive, l'API Voice Interaction Service fournit une abstraction pour intégrer des assistants vocaux dans les applications, permettant aux utilisateurs de contrôler des fonctions comme la lecture musicale ²³⁴³²⁰. Ces systèmes représentent une évolution majeure de l'interface de commande, passant d'une interaction manuelle à une interaction conversationnelle, où l'intention de l'utilisateur est extraite de la parole et traduite en une séquence d'instructions exécutables par le système.

En conclusion, les télécommunications et les interfaces physiques démontrent que la commande n'est pas intrinsèquement liée à un clavier. Que ce soit à travers les tonalités DTMF, les chaînes USSD, ou les interactions vocales, l'information est codée dans des signaux physiques ou acoustiques qui sont ensuite interprétés par des systèmes informatiques pour déclencher des actions. Ces formes de commandes sont non seulement exécutables mais aussi fondamentales pour l'interaction humain-machine dans de nombreux domaines de la vie quotidienne.

Robotique, Aéronautique et Électronique Embarquée : Pilotes de Systèmes Autonomes

Le domaine de la robotique, de l'aéronautique et de l'électronique embarquée est un bastion de commandes spécialisées, conçues pour le contrôle précis, en temps réel et souvent à distance, de systèmes autonomes complexes. Les dispositifs concernés vont des drones micro-aériens (UAV) aux bras robotiques industriels, en passant par les contrôleurs embarqués dans des véhicules ou des machines de production. Les langages

et protocoles utilisés dans ce secteur sont caractérisés par leur efficacité, leur faible latence et leur robustesse dans des environnements hétérogènes.

Le protocole MAVLink (Micro Air Vehicle Link) est devenu l'interconnexion de facto pour les systèmes de vol à voilure tournante et les autres véhicules autonomes (UAVs, UGVs, USVs) ^{20 335}. C'est un protocole de communication léger, conçu pour fonctionner sur des liens radio à faible bande passante et asymétriques, typiques des UAVs ^{75 76}. Il transporte des messages courts contenant des informations de capteurs (position GPS, orientation, vitesses), des commandes de vol et des informations de statut ³². La composante "commande" de MAVLink est particulièrement importante ; elle est constituée d'un ensemble de messages MAV_CMD qui représentent des actions prédéfinies ³³. Par exemple, un opérateur au sol peut envoyer un message COMMAND_LONG avec le code MAV_CMD_NAV_TAKEOFF (identifiant 241) pour commander au drone de décoller, ou MAV_CMD_DO_SET_SERVO (identifiant 223) pour positionner un servomoteur à un canal et à une position spécifiques ^{11 34 232}. Le protocole supporte jusqu'à 7 paramètres pour chaque commande, permettant une grande flexibilité ¹⁵. Les messages sont encapsulés dans des trames comprenant un identifiant de message, des données, et une somme de contrôle pour la vérification de l'intégrité ¹⁴. Des bibliothèques comme PyMAVLink pour Python ou MavLinkCom pour C++ simplifient grandement l'implémentation de stations de sol ou de pilotes autonomes en gérant la sérialisation/désérialisation des messages ^{12 141}. La version 2 de MAVLink introduit des capacités supplémentaires, comme la possibilité d'utiliser des ID de message supérieurs à 255 pour des messages personnalisés, ce qui est crucial pour les applications spécialisées ⁹⁴.

Au-delà des protocoles de communication génériques, de nombreux systèmes robotiques ont leurs propres langages de contrôle natifs, optimisés pour la performance et la précision. Universal Robots, un leader des robots collaboratifs, utilise le langage URScript. C'est un langage de script de haut niveau qui permet de contrôler directement le bras robotique, en définissant des mouvements, des vitesses, des accélérations, et en interagissant avec les entrées/sorties numériques et analogiques du robot ^{95 143}. Un script URScript peut, par exemple, commander au robot de suivre une trajectoire circulaire tout en lisant l'état d'un capteur de proximité et en ajustant sa vitesse en conséquence. Pour utiliser URScript, le robot doit être mis en mode "remote control", ce qui permet à un ordinateur externe de lui envoyer des snippets de code en temps réel ³³⁰³³¹. De même, les contrôleurs FANUC utilisent le langage KAREL, une sorte de Pascal conçu pour la programmation de contrôleurs industriels, permettant des tâches complexes de synchronisation, de communication et de diagnostic ³⁰².

L'écosystème de la robotique est souvent construit autour d'une architecture logicielle commune, comme le Robot Operating System (ROS). Il est important de noter que ROS n'est pas un système d'exploitation au sens traditionnel, mais plutôt une collection de logiciels libres (bibliothèques et outils) qui fournissent des fonctionnalités de base couramment requises pour la robotique ⁹⁶. ROS expose une interface en ligne de commande (`ros2`) qui sert de point d'entrée central pour interagir avec le système robotique ⁹⁷. À partir de cette CLI, un ingénieur peut lancer et arrêter des nœuds (processus individuels), inspecter la topologie du graphe de communication (voir quels nœuds publient sur quels sujets), appeler des services, et afficher des informations d'état ^{97 230}. Par exemple, la commande `ros2 node list` affiche tous les nœuds en cours d'exécution, tandis que `ros2 topic echo /camera/image_raw` permet d'inspecter les images provenant d'une caméra ⁹⁷. ROS2, la deuxième version, renforce cette capacité avec `ros2_control`, une infrastructure standardisée pour contrôler tout type de matériel robotique, de la même manière que Linux a une couche de virtualisation matérielle (la couche pilote) ¹⁴⁴³³³.

Dans le monde plus large de l'électronique embarquée, où les ressources sont limitées, les commandes prennent souvent la forme d'interfaces en ligne de commande minimalistes. Le chargeur d'amorçage U-Boot (Universal Bootstrapper) est un exemple emblématique. Il s'exécute immédiatement après l'allumage du système, avant même le chargement du noyau Linux, et fournit une console série (UART) avec son propre ensemble de commandes ^{52 262}. Ces commandes permettent de charger des images de noyau et de systèmes de fichiers depuis différentes sources (mémoire flash, carte SD, réseau via TFTP) en mémoire, puis de démarrer le système ¹¹³¹¹⁵. Des commandes comme `setenv` et `printenv` permettent de gérer les variables d'environnement du bootload, `tftpboot` de télécharger un fichier, et `md` de lire la mémoire ¹¹³. Accéder à l'invite U-Boot est essentiel pour le développement, le débogage et la mise à jour des systèmes embarqués, en particulier lorsqu'il n'y a pas de système d'exploitation fonctionnel ³¹²³¹⁷. De même, BusyBox est un utilitaire qui combine de nombreuses fonctions standards de GNU Core Utilities en un seul petit exécutable, fournissant un environnement de type shell minimaliste dans les systèmes Linux embarqués ^{59 116}. Il est courant de trouver une invite BusyBox sur des routeurs, boîtiers TV ou autres appareils Linux minimaux, offrant des commandes de base pour la navigation dans le système de fichiers, la gestion du réseau, etc. ⁶⁰. Ces interfaces CLI, bien que simples, sont des points d'entrée de commande indispensables pour l'administration et la maintenance de ces systèmes discrets.

Domaine	Dispositif/ Technologie	Langage/Protocole	Type de Commande	Exemple de Commande
Robotique (Collaborative)	Bras Robotique Universal Robots	URScript ⁹⁵	Scripts de contrôle de mouvement et d'I/O ¹⁴³	<code>movej (pHome, a=1.396, v=1.05)</code> (mouvement joint)
Aérospatiale (Drone)	UAV (DJI, Parrot, ArduPilot)	MAVLink (messages MAV_CMD) ¹¹	Actions de vol et paramètres ³³	<code>MAV_CMD_NAV_TAKEOFF</code> (Décollage)
Robotique (Logiciel)	Robot (ROS 2)	<code>ros2 CLI</code> ⁹⁷	Gestion de nœuds, services et topics ⁹⁶	<code>ros2 run my_pkg my_node</code>
Électronique Embarquée	Chargeur d'amorçage (U-Boot)	U-Boot CLI ⁵²	Gestion du boot, chargement de firmware ¹¹³	<code>setenv serverip 192.168.1.10</code>
Électronique Embarquée	Système Linux Minimisé	BusyBox Shell ⁵⁹	Commandes UNIX de base (ls, cp, ifconfig) ⁶⁰	<code>ifconfig eth0 192.168.1.50</code>

Ces exemples illustrent que dans les domaines de la robotique et de l'électronique embarquée, la commande est intimement liée à la performance, à la fiabilité et à l'interaction directe avec le matériel. Les protocoles comme MAVLink et les langages natifs comme URScript sont des langages d'instructions dédiés, tandis que les interfaces comme celle d'U-Boot représentent des couches de contrôle fondamentales qui précèdent l'OS.

Frontières de l'Exécution : Langages Quantiques et Biologiques

Pour répondre à l'ambition d'un répertoire véritablement exhaustif, il est impératif d'explorer les domaines où le concept de "langage exécutable" est poussé à ses limites extrêmes, se détachant des architectures de von Neumann traditionnelles pour s'ancrer dans des substrats physiques et moléculaires. Les disciplines de la biologie synthétique et du calcul quantique proposent des modèles radicalement différents de l'instruction, où les "codes" sont des séquences d'ADN ou des impulsions physiques, et les "exécutants" sont des machines à l'échelle atomique ou moléculaire. Ces domaines, bien que techniquement avancés, relèvent fondamentalement du même principe : une information structurée est interprtable comme une intention directive par un système capable de la lire et d'agir en conséquence.

En biologie synthétique, l'ADN est considéré comme un langage d'exécution universel ²³³. Les quatre bases azotées (Adénine, Thymine, Cytosine, Guanine) forment un alphabet

qui, lorsqu'elles sont assemblées en séquences spécifiques, codent pour des protéines. On peut donc voir les gènes comme des programmes et les protéines comme les résultats de leur exécution. Des plateformes comme l'Infobiotics Workbench (IBW) visent à créer un environnement computationnel pour la conception de systèmes biologiques, intégrant des langages de description et des simulateurs ²³³. Des travaux de recherche poussent cette analogie plus loin en créant des circuits logiques entièrement réalisés avec de l'ADN. Des chercheurs ont développé des machines à états finis (FSM) à base d'ADN qui peuvent "se souvenir" d'une séquence d'inputs moléculaires, agissant comme un enregistreur d'historique temporel ¹⁴⁷¹⁴⁸³²¹. D'autres ont implémenté des structures de données comme des piles (stacks) en utilisant des polymères ADN, capables de stocker et de récupérer des informations dans un ordre LIFO (Last-In, First-Out) ³²². Dans ces systèmes, les entrées sont des oligonucléotides spécifiques qui déclenchent des réactions chimiques séquentielles, pilotant ainsi le circuit. Des langages de programmation comme DNA-Coder ont même été développés pour concevoir des circuits ADN à partir de descriptions de haut niveau ¹⁹⁸. L'ARN messenger (ARNm) peut être vu comme un "compilateur" biologique qui transcrit l'instruction codée dans le gène (ADN) en une forme intermédiaire (ARNm) qui sera ensuite traduite en protéine ("exécutable") par le ribosome ²³³. L'ensemble de ce processus est un cycle complet de lecture, compilation et exécution d'un programme naturel.

Le calcul quantique représente une autre frontière, où les bits classiques (0 et 1) sont remplacés par des qubits, des systèmes physiques pouvant exister dans une superposition d'états. La programmation d'un ordinateur quantique ne suit pas le paradigme classique. Au lieu de manipuler des données, on manipule l'état quantique lui-même. Pour cela, des langages d'assemblage quantique ont été créés. OpenQASM (Quantum Assembly Language) de IBM et Quil de Rigetti sont des exemples de tels langages ²³³. Un programme QASM est une liste d'instructions (portes quantiques) qui sont appliquées à des qubits. Chaque porte est une transformation unitaire de l'état du système, représentée par une matrice mathématique. Des portes comme la porte CNOT (Contrôle-NOT), qui intrique deux qubits, ou la porte de rotation, qui modifie l'état d'un seul qubit, sont des "instructions" physiques directes ¹⁸⁹. Un algorithme quantique comme Shor ou Grover est donc exprimé comme une séquence de ces portes. Des cadres de programmation comme Qiskit (pour OpenQASM) et Cirq (pour Quil) fournissent des bibliothèques pour construire, simuler et exécuter ces circuits quantiques sur des ordinateurs quantiques réels ou des simulateurs ¹⁸⁹²³³. Microsoft propose également le langage Q#, qui est un langage de haut niveau plus abstrait, mais qui est compilé en un plan d'opérations sur des qubits ²³³. Dans ce contexte, la "machine" est un dispositif physique complexe (supraconducteur, ion piégé, photonique), et la "commande" est une impulsion laser ou

électromagnétique très précise qui réalise l'opération de porte quantique. Le code n'est donc pas exécuté par un CPU, mais par le hardware quantique lui-même.

Enfin, pour atteindre une exhaustivité totale, le répertoire doit inclure des systèmes qui, bien que non conventionnels, sont fermement ancrés dans la réalité informatique. Les langages esotériques, conçus pour être difficiles à programmer (voire impossibles à utiliser pour la programmation pratique), doivent être pris en compte. Des langages comme Malbolge, dont le but était de créer le langage de programmation le plus difficile au monde, sont des curiosités informatiques, mais ils sont tout à fait exécutables par un interpréteur Malbolge existant ²³³. Leur existence prouve qu'une fois qu'un système est capable de les interpréter, ils deviennent une forme de "commande". De même, les automates cellulaires, comme ceux définis par Stephen Wolfram, sont des systèmes de calcul universels. Une règle d'automate cellulaire, comme la "Règle 30" ou la "Règle 110", est un simple entier qui encode les règles de transition d'un univers à une dimension. À partir d'une graine de départ simple, cette règle auto-exécute une évolution complexe et parfois chaotique, agissant comme un programme complet qui transforme un état initial en un état final ²³³. Bien que ces exemples semblent théoriques ou ludiques, ils démontrent la diversité incroyable des formes que peut prendre une "instruction exécutable". Ils sont tous recevables et exécutables par un système informatisé (même si ce système est un simulateur ou un interpréteur conçu spécifiquement pour eux), ce qui les place pleinement dans le champ de notre enquête. L'ADN, les qubits et les automates cellulaires représentent la quintessence de la commande : une information qui, lorsqu'elle est présente dans un système adéquat, le guide vers un nouvel état.

Domaine Non-Conventional	Substrat Physique/ Moléculaire	Langage/Code	"Exécutant"	Exemple d'Instruction
Biologie Synthétique	Molécules d'ADN/ARN/ Protéines	Séquences d'ADN, ARNm	Machinerie cellulaire (ribosomes, enzymes) ²³³	Séquence codant pour une enzyme de restriction
Calcul Quantique	Qubits (supraconducteurs, ions, photons)	OpenQASM, Quil, Q#	Dispositif physique quantique	Porte CNOT (intrication de deux qubits) ¹⁸⁹
Automates Cellulaires	Grille de cellules discrètes	Règle de Wolfram (entier)	Algorithme de simulation de l'automate	Règle 110 (générateur de complexité) ²³³
Langages Esotériques	Ordinateur (via interpréteur)	Malbolge, Brainfuck ²³³	Interpréteur Malbolge / Brainfuck	[] + - < > , . (manipulation de la mémoire)
Hiéroglyphes/ Cunéiforme	Matrice physique (pierre, argile)	Alphabet/ syllabaire encodé	Scribe ou archéologue (lecteur)	Glyphes Hiéroglyphiques ²³³

Ces domaines, bien que distincts, partagent un principe commun : la conversion d'une séquence d'informations symboliques en une action physique ou computationnelle. Ils élargissent notre compréhension de la commande au-delà des limites de l'informatique

classique, nous amenant à reconnaître que le langage de l'instruction est peut-être plus ancien et plus fondamental que ne le suggère l'ordinateur moderne.

Taxonomie et Méta-Modélisation : Structuration du Répertoire NiX-α94

La construction d'un répertoire exhaustif et utile, tel que celui visé par le projet NiX-α94, ne peut se contenter d'une simple liste alphabétique des langages et protocoles. Pour être véritablement "souverain sémantiquement" et alimenter un "méta-compilateur Ouroboros Polyglotte", la taxonomie doit être à la fois technique et sémantique, structurée pour permettre non seulement l'identification mais aussi la classification et l'orchestration des intentions derrière chaque forme d'expression ²³³. La base de données `T00LS_DB` fournie constitue un excellent squelette, mais elle doit être enrichie et agrandie pour refléter la diversité des commandes identifiées dans ce rapport. L'approche recommandée est une combinaison de dimensions techniques pour l'indexation et de dimensions sémantiques pour la classification avancée.

La première étape consiste à formaliser les dimensions techniques, qui permettront de catégoriser chaque élément par son origine et son mode d'exécution.

- **Type** : Cette dimension classe l'élément selon sa nature fondamentale. Elle s'étend au-delà de "language" pour inclure "protocol" (Modbus, MAVLink), "format" (JSON, YAML), "markup" (HTML), "library" (OpenCV, PyTorch), "framework" (ROS, TensorFlow), "tool" (Docker, Terraform), "os" (NixOS), et "hardware" (FPGA, PLC) .
- **Domaine** : Cette dimension indique le champ d'application principal de l'élément. Elle permet de regrouper les technologies par sujet, comme "industriel", "télécom", "robotique", "biologie", "quantique", "ia", "gpu", "systeme", "securite", etc. .
- **Backend(s)** : Cette dimension décrit comment l'élément est exécuté ou intégré. Les valeurs possibles incluent "native" (compilé directement pour la machine), "interpreter" (lu et exécuté par un programme), "compiler" (traduit en code machine), "wasm" (WebAssembly), "ffi" (Foreign Function Interface pour appeler depuis un autre langage), "jvm" (Java Virtual Machine), "dotnet" (Microsoft .NET runtime), "hardware" (synthétisé sur du matériel) .

Ces trois dimensions techniques, inspirées de la structure `T00LS_DB`, permettent de créer un index riche et multi-facettes pour rechercher et identifier n'importe quelle technologie.

La deuxième étape, et la plus ambitieuse, est de développer une taxonomie sémantique. Cette approche vise à capturer l'« atome d'intention » de chaque commande, indépendamment de sa manifestation formelle ²³³. Cela permettrait au méta-compilateur JGNL-SKU de traiter un langage de haut niveau, un protocole de bas niveau et un code ADN comme des entités de même nature : des porteurs d'intention.

- **Intention** : Cette dimension décrit le but de la commande. Elle pourrait inclure des valeurs comme ``control`` (contrôle d'un appareil ou d'un processus, ex: URScript, Modbus write), ``query`` (interrogation d'un état, ex: SCPI ``:SYST:ERR?``, SQL ``SELECT``), ``computation`` (calcul d'une valeur, ex: Julia, Fortran), ``data_exchange`` (transmission de données structurées, ex: MQTT publish), ``configuration`` (définition du comportement d'un système, ex: YAML config file), ``physical_action`` (déclenchement d'un événement physique, ex: DTMF tone, PWM signal), et ``biological_instruction`` (programme d'action moléculaire, ex: séquence d'ADN).
- **Interface Type** : Cette dimension décrit le mode d'interaction. Elle va de l'interface en ligne de commande (shell, CLI) et l'interface graphique (GUI, HMI) aux API (REST, gRPC), aux connexions série (UART, RS-485), sans fil (Wi-Fi, Bluetooth, Radio MAVLink), vocales (Siri, Tesla voice control) ou tactiles (écran tactile, boutons).
- **Execution Level** : Cette dimension qualifie le niveau d'abstraction de l'exécution. Elle va du code machine (native) et de l'assembleur (ASM) au code de haut niveau (Python, C++) en passant par les paradigmes déclaratifs (SQL, HTML/CSS), logiques (Prolog), fonctionnels (Haskell), et physiques (impulsions quantiques, séquences ADN).

En combinant ces dimensions, on peut créer des entrées de base de données riches et interconnectées. Par exemple, la commande `MAV_CMD_NAV_TAKEOFF` serait classée comme un `protocol` (MAVLink) dans le `domaine` "robotique/aérospatiale", avec un `backend` "wireless", une `intention` "control", une `interface_type` "CLI/API", et un `execution_level` "high-level".

Le tableau suivant illustre cette approche de taxonomie hybride pour plusieurs exemples clés :

Nom	Type	Domaine	Backends	Intention	Interface Type
Python	language	general, ia, scripting	interpreter	computation, automation	shell, api, cli
Modbus	protocol	industriel	serial, tcp	control, query	serial, network
SCPI	protocol	industriel, science	serial, gpib, tcp	control, query	serial, network
MAVLink	protocol	robotique, aérospatiale	wireless, serial	control, telemetry	wireless, network
URScript	language	robotique	native	control	remote_api
G-code	language	industriel, fabrication	native	control	serial
DTMF	signaling	telecom	telephony	control	physical_input
USSD	protocol	telecom	cellular	query, transaction	cellular
OpenQASM	language	quantique	hardware	computation	api
ADN	code	biologie	biological	biological_instruction	molecular
Assembly	language	systeme	native	control	shell
JSON	format	data	parser	data_exchange	api, file

Enfin, pour structurer le contenu, il est conseillé de créer des sections thématiques dans le répertoire, comme celles explorées dans ce rapport : "Commandes Industrielles", "Commandes Télécom", "Commandes Robotiques", "Frontières de l'Exécution", etc. Dans chaque section, les technologies sont présentées avec une description de leur rôle et de leur mode d'interaction. Pour les protocoles comme Modbus ou SCPI, il serait judicieux de créer une entrée principale décrivant le protocole lui-même, qui pointerait vers des sous-entrées décrivant les commandes spécifiques les plus courantes (par exemple, une entrée `Modbus_Write_Coil`, une autre `SCPI_Reset_Instrument`). Cette approche évite la duplication et facilite la maintenance.

En somme, la réalisation du répertoire NiX-α94 est un projet monumental qui nécessite une double approche. D'une part, une collecte exhaustive et rigoureuse des technologies existantes, en s'appuyant sur des standards documentés et des pratiques industrielles éprouvées. D'autre part, une abstraction sémantique profonde visant à découvrir les principes unificateurs sous-jacents à la diversité des langages d'instruction. En adoptant une taxonomie hybride qui intègre à la fois les dimensions techniques de l'implémentation et les dimensions sémantiques de l'intention, il est possible de construire non pas un simple catalogue, mais un véritable atlas de l'instruction, un outil capable de naviguer dans l'immense paysage de la commande informatique, de l'invite de commande du terminal à la machinerie vivante de la cellule.

Référence

1. Connecting Touch Screens to Industrial Machines! - YouTube <https://www.youtube.com/watch?v=rQ9UwKRThJQ>
2. Free Low-Cost HMI Software for Modbus/TCP PLC Communication https://industrialmonitordirect.com/blogs/knowledgebase/free-low-cost-hmi-software-for-modbustcp-plc-communication?srsltid=AfmBOooB_rzdBbK4J-xtnlrj5ytUtnpQdb79obYPQmUfVARJ9V47Xs8h
3. Build a FREE HMI for BRX PLC Using Modbus TCP in Minutes - YouTube <https://www.youtube.com/watch?v=1TjsdLwTKM0>
4. How to Use Command Line Interface & SCPI for Scripting (Introduction) <https://kb.veexinc.com/en/knowledge/how-to-use-command-line-interface-scp-for-scripting-introduction>
5. Modbus SCADA HMI Systems | ICP DAS USA Inc - Data Acquisition <https://www.icpdas-usa.com/modbus-scada-hmi-systems.html?srsltid=AfmBOopdXa-8jksRDG10ptu5kW1lRdBO3XHTsg20YBFVcfnLXGXfYA2F>
6. Sending Text from my PLC to HMI with MODBUS connection - AdvancedHMI <https://advancedhmi.com/forum/index.php?topic=2907.0>
7. Communicate with instruments via SCPI commands <https://community.se.com/t5/Ask-the-Community/Communicate-with-instruments-via-SCPI-commands/td-p/471274>
8. Basic programming for PLC & Human Machine Interface (HMI) - YouTube <https://www.youtube.com/watch?v=VV8Mqb1tw5w>
9. nModbus (C#) for Windows - ICP DAS https://www.icpdas.com/en/product/guide+Software+Development__Tools+Modbus__Tool
10. Modbus - Clarify Docs <https://docs.clarify.io/developers/industrial-protocols/modbus>
11. Command Protocol - MAVLink Guide <https://mavlink.io/en/services/command.html>
12. Questions about mavlink commands and the pymavlink documentation <https://discuss.bluerobotics.com/t/questions-about-mavlink-commands-and-the-pymavlink-documentation/11446>
13. Easiest way to submit a longer non standard character string via MAVLink <https://robotics.stackexchange.com/questions/7002/easiest-way-to-submit-a-longer-non-standard-character-string-via-mavlink>

14. MAVLink Messages | Dissecting the Protocol - YouTube <https://www.youtube.com/watch?v=Ha66uKC-od0>
15. PyMAVLink In A Nutshell - DarkSleep <https://www.darksleep.com/mavlink/pymavlink/>
16. MAVLink Basics — Dev documentation - ArduPilot <https://ardupilot.org/dev/docs/mavlink-basics.html>
17. Mavlink Manual Mode rejected - ANAFI USA / Thermal / 4K <https://forum.developer.parrot.com/t/mavlink-manual-mode-rejected/19253>
18. PLAY_TUNE message should be non-ambiguous #1251 - GitHub <https://github.com/mavlink/mavlink/issues/1251>
19. problem with using mavlink library - Google Groups <https://groups.google.com/g/mavlink/c/mRmcniPbHVg>
20. MAVLink Support - MATLAB & Simulink - MathWorks <https://www.mathworks.com/help/uav/mavlink-support.html>
21. I ranked EVERY terminal emulator. It was nuts - YouTube <https://www.youtube.com/watch?v=WxzYtdIcHnQ>
22. Can you provide examples of operating systems that use shells ... <https://www.quora.com/Can-you-provide-examples-of-operating-systems-that-use-shells-other-than-bash>
23. cdleon/awesome-terminals: Terminal Emulators - GitHub <https://github.com/cdleon/awesome-terminals>
24. Suggestions for Terminal Emulators on Linux, Windows and Mac [closed] <https://serverfault.com/questions/7807/suggestions-for-terminal-emulators-on-linux-windows-and-mac>
25. Terminal Emulators <https://terminaltrove.com/terminals/>
26. Unix Shells and Terminals - Erik Explores - Substack <https://erikexplores.substack.com/p/unix-shells-and-terminals>
27. List of terminal emulators - Wikipedia https://en.wikipedia.org/wiki/List_of_terminal_emulators
28. A Guide to the Terminal, Console, and Shell - The Valuable Dev <https://thevaluable.dev/guide-terminal-shell-console/>
29. Terminals, shells, consoles, and command lines - Red Hat <https://www.redhat.com/en/blog/terminals-shells-consoles>
30. A list of terminal emulators available on Linux Any favorite? - Facebook <https://www.facebook.com/groups/linux.commands/posts/7982681171849387/>
31. MAVLINK Common Message Set - Hamish Willee https://hamishwillee.gitbooks.io/ham_mavdevguide/en/messages/common.html

32. MAVLink Messaging | PX4 Guide (main) <https://docs.px4.io/main/en/mavlink/index>
33. MAVLink - TritonUAS Wiki <https://tritonuas.github.io/wiki/electrical/communication/wired/mavlink/>
34. common-mavlink-mission-command-messages-mav_cmd.rst https://github.com/ArduPilot/ardupilot/wiki/blob/master/common/source/docs/common-mavlink-mission-command-messages-mav_cmd.rst
35. Samsung SmartThings API is getting rid of free access https://www.reddit.com/r/homeassistant/comments/1v22add/samsung_smarththings_api_is_getting_rid_of_free/
36. Embedded Systems/Common Protocols https://en.wikibooks.org/wiki/Embedded_Systems/Common_Protocols
37. Understanding input types in Shortcuts on iPhone or iPad <https://support.apple.com/guide/shortcuts/input-types-apd7644168e1/ios>
38. What's New in Shortcuts for iOS 26 - Matthew Cassinelli <https://matthewcassinelli.com/whats-new-in-shortcuts-for-ios-ipados-macos-watchos-visionos-26/>
39. Pointers about portable shell programming <https://www.in-ulm.de/~mascheck/various/portability/>
40. Sentry Mode https://www.tesla.com/ownersmanual/model3/en_us/GUID-56703182-8191-4DAE-AF07-2FDC0EB64663.html
41. Model 3 – Sports Electric Sedan <https://www.tesla.com/model3>
42. IEC 61131-3 defines 5 PLC programming languages - LinkedIn https://www.linkedin.com/posts/lahcen-elbouhali-a8a668273_types-of-plc-programming-languages-in-industrial-activity-7373692203199524865-pUs8
43. IEC 61131-3 and PLCopen <https://www.plcopen.org/standards/logic/iec-61131-3/>
44. Structured Text PLC Programming: IEC 61131-3 Complete Guide <https://industrialmonitordirect.com/blogs/knowledgebase/structured-text-plc-programming-iec-61131-3-complete-guide?srsltid=AfmBOooBzsCYRkoazH11rOdMqkZc7nwvaGvMnPMiUkyUfFwnVHHTGhcD>
45. Why is the instruction list (IL) language for PLCs falling out of favor? <https://www.motioncontroltips.com/why-is-the-instruction-list-il-language-for-plcs-falling-out-of-favor/>
46. IEC 61131-3 - Wikipedia https://en.wikipedia.org/wiki/IEC_61131-3
47. The 5 IEC 61131-3 Languages Compared | PLCcode.ai <https://plccode.ai/iec-61131-3>
48. 3.1 IEC 61131-3 Programming Overview (IEC 61131-3 Basics with ... <https://www.youtube.com/watch?v=iaj1PefPkEk>
49. IEC 61131-3:2003 - Programmable Controllers Programming Languages <https://standards.iteh.ai/catalog/standards/iec/34e7370d-d9cf-4237-a2b4-ff7fcd2b8c94/>

iec-61131-3-2003?srsltid=AfmBOopPKydbnFUC8IbJl2G4EM8XUs1gGj5RyHkL5_M0G-HRiZp32lsD

50. How do IEC 61131-3 languages rank in terms of execution speed? - Reddit https://www.reddit.com/r/PLC/comments/8hljj9/how_do_iec_611313_languages_rank_in_terms_of/
51. Overview of IEC 61131-3 in Industrial Automation Systems - Control.com <https://control.com/technical-articles/an-overview-of-iec-61131-3-Industrial-Automation-Systems/>
52. U-Boot Reference Manual <https://hub.digi.com/dp/path=/support/asset/u-boot-reference-manual/>
53. How do I connect to the linux shell of an ARM processor via ... https://www.reddit.com/r/embedded/comments/zyrcc0/how_do_i_connect_to_the_linux_shell_of_an_arm/
54. linux - u-boot flash emmc ttl serial <https://stackoverflow.com/questions/28042023/u-boot-flash-emmc-ttl-serial>
55. Tutorial: Introduction to the Embedded Boot Loader U-boot ... <https://www.youtube.com/watch?v=INWghYZH3hI>
56. analogdevicesinc/u-boot-sharc: Uboot repository ... <https://github.com/analogdevicesinc/u-boot-sharc>
57. Building a Tiny CLI Shell for Tiny Firmware - Interrupt - Memfault <https://interrupt.memfault.com/blog/firmware-shell>
58. 3.1. U-Boot — Processor SDK Linux Documentation https://software-dl.ti.com/processor-sdk-linux/esd/docs/06_03_00_106/linux/Foundational_Components_U-Boot.html
59. BusyBox - The Swiss Army Knife of Embedded Linux <https://busybox.net/downloads/BusyBox.html>
60. Basic Linux Commands <https://developer.toradex.com/software/linux-resources/basic-linux-commands/>
61. Embedded Command Line Interfaces Will Change Your Life ... <https://www.youtube.com/watch?v=zW5sHGd1jlc>
62. Telit_80617ST10991A_AT-Commands-Reference-Guide.pdf https://multitech.com/wp-content/uploads/Telit_80617ST10991A_AT-Commands-Reference-Guide.pdf
63. TelephonyManager | API reference <https://developer.android.com/reference/android/telephony/TelephonyManager>
64. Fish Shell: Great features, but not POSIX : r/programming https://www.reddit.com/r/programming/comments/1i0a8w7/fish_shell_great_features_but_not_posix/
65. Supported Shells - Warp docs <https://docs.warp.dev/getting-started/supported-shells/>

66. List of programming languages by type https://en.wikipedia.org/wiki/List_of_programming_languages_by_type
67. First-class support for interactive use in the language? <https://internals.rust-lang.org/t/first-class-support-for-interactive-use-in-the-language/20803>
68. Interactive programming (repl) [https://rosettacode.org/wiki/Interactive_programming_\(repl\)](https://rosettacode.org/wiki/Interactive_programming_(repl))
69. List Of Programming Languages By Type - My Docs - GitBook <https://bryan-guner.gitbook.io/my-docs/programming-languages/programming-languages/list-of-programming-languages-by-type>
70. Introduction to Modbus <https://www.modbus.org/introduction-to-modbus>
71. How to set USB port as RS-485 entrance? How to interpret ... <https://control.com/forums/threads/how-to-set-usb-port-as-rs-485-entrance-how-to-interpret-growatt-inverter-commands.50735/>
72. Digital Data Communication Theory <https://control.com/textbook/digital-data-acquisition-and-networks/digital-data-communication-theory/>
73. Legal Code | Creative Commons Attribution License <https://control.com/textbook/creative-commons-attribution-license/legal-code/>
74. The terminal, the console and the shell - what are they? - Unix Digest <https://unixdigest.com/articles/the-terminal-the-console-and-the-shell-what-are-they.html>
75. MAVLink Developer Guide <https://mavlink.io/en/>
76. MAVLink - Marshalling / communication library for drones. <https://github.com/mavlink/mavlink>
77. Genomic messaging system language including command ... <https://pubmed.ncbi.nlm.nih.gov/15822903/>
78. Procedure Examples of T1059 Command and Scripting ... <https://www.picussecurity.com/resource/blog/t1059-command-and-scripting-procedure-examples>
79. MavLink Tutorial for Absolute Dummies (Part –I) <https://discuss.px4.io/uploads/short-url/vS0JJd3BQfN9uF4DkY7bAeb6Svd.pdf>
80. Spectrum Lab Interpreter <https://www.qsl.net/dl4yhf/speclab/interpr.htm>
81. MAVLink <https://en.wikipedia.org/wiki/MAVLink>
82. Best Free HMI Software for Modbus/TCP | Comparison Guide https://industrialmonitordirect.com/blogs/knowledgebase/free-low-cost-hmi-software-for-modbustcp-plc-communication?srsId=AfmBOoo40OqPV_r3cjZr5HfeUhI0rhRcrPyccPGBF1S5ZDkmegqprFCz

83. Tips & Tricks: Mastering Modbus with AVEVA Edge Software <https://www.icpdas-usa.com/Tips-Tricks-Mastering-Modbus-with-AVEVA-Edge-Software?srsltid=AfmBOoqdiV80INAofdWEtfsiphvqeHVGsTkHcBNvQCh4ww9uJ-obfCDS>
84. PLC & HMI Programming Services <https://is-inc.com/services/plc-hmi-programming/>
85. Standard Commands for Programmable Instruments https://en.wikipedia.org/wiki/Standard_Commands_for_Programmable_Instruments
86. 2. Introducing SCPI Commands https://www.rohde-schwarz.com/us/driver-pages/remote-control/remote-programming-environments_231250.html
87. SCPI Syntax and Usage <https://www.ni.com/docs/en-US/bundle/wireless-test-system-api-overview/page/scpi-usage.html?srsltid>
88. SCPI Basics https://helpfiles.keysight.com/csg/n5106a/scpi_basics.htm
89. SCPI, A Programming Language for Measurement Device <https://filpal.wordpress.com/2021/06/27/scpi-a-programming-language-for-measurement-device/>
90. Edge Driver Architecture and Matter Support (technical ... <https://community.smarthings.com/t/edge-driver-architecture-and-matter-support-technical-discussion/296011>
91. How to set up Matter device with Alexa, Google, Apple or ... <https://www.tp-link.com/uk/support/faq/3564/>
92. How to add any Matter Smart Home Devices ... <https://blog.matterhubs.com/add-matter-devices-to-smart-home-systems/>
93. Set up Mavlink connection and serialize messages <https://nl.mathworks.com/matlabcentral/answers/2178393-set-up-mavlink-connection-and-serialize-messages>
94. MAVLink Versions https://mavlink.io/en/guide/mavlink_version.html
95. Sending URScript code to the robot https://docs.ros.org/en/jazzy/p/ur_robot_driver/doc/usage/script_code.html
96. Programming Multiple Robots with ROS 2 https://osrf.github.io/ros2multirobotbook/ros2_cli.html
97. Overview and the most important commands of ROS2 https://wiki.hshl.de/wiki/index.php/Overview_and_the_most_important_commands_of_ROS2
98. An alternative to touchscreens? In-car voice control is ... <https://arstechnica.com/cars/2023/02/voice-assistants-in-cars-are-finally-getting-good-enough-to-rely-on/>
99. Voice Control In Cars: Where Are We Headed? <https://www.readspeaker.com/blog/voice-control-car/>
100. Programming with SCPI https://dl.cdn-anritsu.com/en-us/test-measurement/ohs/10450-00075H/MS2070A-MS2080A-Field_Master_Help/scpi-remote-programming.52.1.html

101. SCPI commands explained for automated testing <https://encida.com/news-and-insights/introduction-to-scp-commands/>
102. A Scalable Approach to Modeling on Accelerated ... <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2022.884128/full>
103. A Scalable Approach to Modeling on Accelerated ... - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC9157770/>
104. C. Rationale for Shell and Utilities https://pubs.opengroup.org/onlinepubs/9799919799/xrat/V4_xcu_chap01.html
105. The Single UNIX Specification V5 (2024) - Introduction <https://www.unix.org/overview.html>
106. 2. Shell Command Language https://pubs.opengroup.org/onlinepubs/9799919799/utilities/V3_chap02.html
107. IEC 61131-3:2003 - Programming languages https://standards.iteh.ai/catalog/standards/iec/34e7370d-d9cf-4237-a2b4-ff7fcd2b8c94/iec-61131-3-2003?srsId=AfmBOopjkPjkjr5BG79yKHNGs5Nkq4ss_vOH1hxgEAxk3p5LFC8n03aH
108. IEC 61131-3 Protocol Overview <https://www.rtautomation.com/technologies/control-iec-61131-3/?srsId=AfmBOorCEkKCFcE8UyB-cU4ZZvN7jjkpUcTi5Dp5KGets00FwAMNLqDQ>
109. IEC 61131-3 Choosing a Programming Language <https://www.dmcinfo.com/blog/30145/iec-61131-3-choosing-a-programming-language/>
110. IEC 61131-3 PLC Programming Standard Explained https://industrialmonitordirect.com/blogs/knowledgebase/iec-61131-3-plc-programming-standard-explained?srsId=AfmBOop_6Hk_hEfSVz3_CNfhqNfWNhR4DNpXEz5UG7bFm5iiNpdjSRDG
111. PLC Programming Languages: IEC 61131-3 Examples ... <https://plcprogramming.io/blog/plc-programming-languages-complete-guide>
112. Booting Linux with U-Boot on QEMU ARM - Freedom Embedded <https://balau82.wordpress.com/2010/04/12/booting-linux-with-u-boot-on-qemu-arm/>
113. Embedded Linux(Part-7): Deep Dive into U-boot <https://www.youtube.com/watch?v=bpaOCnnHxHk>
114. Embedded Linux | Introduction To U-Boot | Beginners <https://www.youtube.com/watch?v=rVaiLgXccSE>
115. How U-boot loads Linux kernel <https://krinkinmu.github.io/2023/08/21/how-u-boot-loads-linux-kernel.html>
116. Linux on AT91SAM9261EK Uboot, Busybox stdout/stderr ... <https://stackoverflow.com/questions/19666696/linux-on-at91sam9261ek-uboot-busybox-stdout-stderr-setup>

117. Understanding the (Embedded) Linux boot process <https://kleinembedded.com/understanding-the-embedded-linux-boot-process/>
118. Booting an Embedded OS: the Booting and U-Boot Phase <https://serhack.me/articles/os-embedded-booting-phase-uboot/>
119. MODBUS over serial line specification and implementation ... <https://www.modbus.org/file/secure/modbusoverserial.pdf>
120. How-To: Building a PLC Project with Modbus Communication <https://control.com/technical-articles/how-to-building-a-plc-project-with-modbus-communication/>
121. MODBUS ASCII Protocol Standard / Definition or .NET ... <https://control.com/forums/threads/modbus-ascii-protocol-standard-definition-or-net-library.55321/>
122. different manufacturers, same Modbus RTU syntax? <https://control.com/forums/threads/different-manufacturers-same-modbus-rtu-syntax.50200/>
123. communication with a ModBus instrument via RS232 <https://control.com/forums/threads/communication-with-a-modbus-instrument-via-rs232.12686/>
124. Making Modbus device communicate with a PC <https://control.com/forums/threads/making-modbus-device-communicate-with-a-pc.27478/>
125. Communication problems , modbus timeout error <https://control.com/forums/threads/communication-problems-modbus-timeout-error.48900/>
126. Modbus communication <https://control.com/forums/threads/modbus-communication.707/>
127. Modbus ??UDP?? <https://control.com/forums/threads/modbus-udp.10884/>
128. The fish language — fish-shell 4.8.1 documentation <https://fishshell.com/docs/current/language.html>
129. Best Free HMI Software for Modbus/TCP | Comparison Guide <https://industrialmonitordirect.com/blogs/knowledgebase/free-low-cost-hmi-software-for-modbustcp-plc-communication?srsltid=AfmBOoqKfxSfy2q0N9Ob12cetX3ewXkDBBX5tjrYK-dLw33sPdItvGMr>
130. Connecting Advanced HMI to a Modbus TCP Server - Quick ... <https://www.youtube.com/watch?v=v2TUBhbJN1Y>
131. SCPI Parser Console <https://helpfiles.keysight.com/csg/NA520xA/Programming/SCPIParserConsole.htm>
132. favalex/modbus-cli: Command line tool to access ... <https://github.com/favalex/modbus-cli>
133. Modbus SCADA HMI Systems https://www.icpdas-usa.com/modbus-scada-hmi-systems.html?srsltid=AfmBOoobJ0cjOQx5mEmfGUr2UNymiSzGuhxBFT7GSK1sk43U0U_MWoYm

134. NMEA Tools https://opencpn.org/wiki/dokuwiki/doku.php?id=opencpn:supplementary_software:nmea_instruments
135. Modbus TCP/IP Industrial Ethernet Protocol Software <https://www.nxp.com/design/design-center/software/development-software/software-for-industrial-networking/modbus-tcp-ip-industrial-ethernet-protocol-software:MODBUS-TCP-INDUSTRIAL-COMMUNICATIONS-SOFTWARE>
136. Using modbus-cli to Read and Write Modbus Coils in LabShock <https://alhasawi.medium.com/using-modbus-cli-to-read-and-write-modbus-coils-in-labshock-e7ff44b93d4c>
137. DTMF commands decoding but not processing? - Archive <https://community.allstarlink.org/t/dtmf-commands-decoding-but-not-processing/19315>
138. DTMF decoder - CCS :: View topic <https://www.ccsinfo.com/forum/viewtopic.php?p=6406>
139. AltusConsulting/dnacentrcli - Cisco Code Exchange <https://developer.cisco.com/codeexchange/github/repo/AltusConsulting/dnacentrcli/>
140. deserializemsg - Deserialize MAVLink message from binary buffer <https://www.mathworks.com/help/uav/ref/mavlinkdialect.deserializemsg.html>
141. MavLinkCom - AirSim <https://microsoft.github.io/AirSim/mavlinkcom/>
142. The Best MacOS Terminal Emulation Programs for ... <https://thenewstack.io/the-best-macos-terminal-emulation-programs-for-developers/>
143. script_code.rst https://github.com/UniversalRobots/Universal_Robots_ROS2_Driver/blob/main/ur_robot_driver/doc/usage/script_code.rst
144. ROS 2 API <https://husarion.com/manuals/lynx/ros2-api/>
145. PLC Communication Protocols Documentation Reference ... <https://industrialmonitordirect.com/blogs/knowledgebase/plc-communication-protocols-documentation-reference-guide?srsltid>
146. Types of Communication Protocols in Control Systems <https://www.forbesmarshall.com/instrupedia/types-of-communication-protocols/>
147. A localized DNA finite-state machine with temporal resolution <https://pmc.ncbi.nlm.nih.gov/articles/PMC8956261/>
148. A localized DNA finite-state machine with temporal resolution <https://asu.elsevierpure.com/en/publications/a-localized-dna-finite-state-machine-with-temporal-resolution>
149. A localized DNA finite-state machine with temporal resolution <https://ui.adsabs.harvard.edu/abs/2022SciA....8M9530L/abstract>

150. Matter/Apple Home Bridge Connecting to an Open ... <https://www.facebook.com/LtechLEDGlobal/videos/matterapple-home-bridge%EF%B8%8Fconnecting-to-an-open-ecosystemnative-support-for-the-ma/870724889240473/>
151. PLC Analog Input & Output Basics – A/D Conversion and ... <https://www.youtube.com/watch?v=bnbsNfFzUMw>
152. How to Program PLC Analog Sensors: Scale, Compare, PID <https://industrialmonitordirect.com/blogs/knowledgebase/plc-analog-sensor-programming-instructions-and-implementation?srsId=AfmBOopif7kb6wTSfHLvsNLx6ETzRxZkdfxEhTBesxKay4vVJ55oldMB>
153. What are Analog Inputs? - Analog Signals Processing in PLC <https://instrumentationtools.com/analog-input-signals-processing-in-plc/>
154. WWDC26: What's new in Shortcuts | Apple <https://www.youtube.com/watch?v=mUoWMIZXQ28>
155. Voice Commands | Tesla Support <https://www.tesla.com/support/voice-commands>
156. Tesla Voice Commands: The COMPLETE List <https://www.youtube.com/watch?v=UmWHcR1wpLQ>
157. Vehicle Commands | Tesla Fleet API <https://developer.tesla.com/docs/fleet-api/endpoints/vehicle-commands>
158. Are Linux commands similar to Mac OS X terminal language? <https://www.facebook.com/groups/GNUAndLinux/posts/10160684368365019/>
159. ushahidi/ussd-engine: Session engine for interaction with ... <https://github.com/ushahidi/ussd-engine>
160. MT8870 DTMF Decoder Module - Arduino Test with PWM ... https://www.youtube.com/watch?v=Wx6C4k_xxz0
161. 3.8.1 - FAIR's documentation! https://fairino-doc-en.readthedocs.io/3.8.1/RobotCommunication/cnde_intro.html
162. Enhancing MAVLink Security in UAV Systems: ... <https://repositorio.uniandes.edu.co/bitstreams/abc2cb38-5c3f-464c-97cb-55e38ae1b1d9/download>
163. DTMF decoder library - Audio <https://forum.arduino.cc/t/dtmf-decoder-library/118979>
164. DSpace - EUI Cadmus - European University Institute <https://cadmus.eui.eu/items/version/33>
165. OSF <https://osf.io/mz637>
166. ANAFI Mavlink protocol version - Parrot Developers Forum <https://forum.developer.parrot.com/t/anafi-mavlink-protocol-version/13780>
167. Structured Text Programming: IEC 61131-3 Platform Guide <https://industrialmonitordirect.com/blogs/knowledgebase/structured-text-programming->

iec-61131-3-platform-guide?srsltid=AfmBOoqtiU49cOu_yuh0NqlB7I6-gZACd1BwMIXwMtMCVgLjT5xW4X6W

168. IEC 61131-3 Structured Text (ST) <https://www.fernhillsoftware.com/help/iec-61131/structured-text/index.html>
169. Ladder Diagram vs Structured Text: All IEC 61131-3 ... <https://www.youtube.com/watch?v=67H6MALcQOQ>
170. IEC 61131-3 Protocol Overview https://www.rtautomation.com/technologies/control-iec-61131-3/?srsltid=AfmBOorExUhX5B_qr8kBtTMQiD-WYiUy9J9T8jGINwNhqgH1OyEPrB-5
171. IEC 61131-3 programming: Where's structured text more ... <https://www.motioncontroltips.com/iec-61131-3-programming-are-there-uses-for-which-structured-text-is-more-appropriate-than-ladder-logic/>
172. IEC 61131-3 Standard: The Core Foundation of Modern ... <https://folksautomation.com/en/blog/software-en/plc-programming-languages-according-to-the-iec-61131-3-standard>
173. In-Depth Comparison of IEC 61131-3 Programming ... <https://feelaautom.fr/en/blog/in-depth-comparison-of-iec-61131-3-programming-languages-ladder-st-fbd-sfc>
174. Easy way to fish shell - DEV Community <https://dev.to/ps173/easy-way-to-fish-shell-l7c>
175. system <https://pubs.opengroup.org/onlinepubs/9799919799/functions/system.html>
176. System Programming in Linux: A Hands-On Introduction <https://dokumen.pub/system-programming-in-linux-a-hands-on-introduction.html>
177. Using DTMF Grammar - Contact Center Express <https://developer.cisco.com/docs/contact-center-express/using-dtmf-grammar/>
178. What is Dual Tone Multi Frequency? Technology Guide <https://sycurio.com/knowledge/glossaries/dual-tone-multiple-frequency-dtmf>
179. ES 201 235-1 - V1.1.1 https://www.etsi.org/deliver/etsi_es/201200_201299/20123501/01.01.01_60/es_20123501v010101p.pdf
180. All about FreeSWITCH (and how it compares with Asterisk) <https://info.teledynamics.com/blog/all-about-freeswitch>
181. Asterisk Docs <https://asterisk.phreaknet.org/>
182. Industrial communication protocols explained | Eaton PSEC https://www.youtube.com/watch?v=_btpMwO2ZvE
183. PLC Communication Protocols Documentation Reference ... <https://industrialmonitordirect.com/blogs/knowledgebase/plc-communication-protocols-documentation-reference-guide?srsltid=AfmBOoryBcvigyWYEKGSL9oZDlxeNKMYjV1rnIsWf2MhBnHqlN6wVhbl>

184. Embedded Programming: PWM & analogWrite() - YouTube <https://www.youtube.com/watch?v=6gqsIeIGJoo>
185. How Pulse Width Modulation (PWM) Works + Example - YouTube <https://www.youtube.com/watch?v=JI8Gyz2auAo>
186. [PDF] SCPI Programmer's Manual for the DT8837 <https://files.digilent.com/manuals/DT8837-SCPI-Programming.pdf>
187. PWM programming | youyeetoo wiki <https://wiki.youyeetoo.com/en/r1/DCpwm>
188. SCPI Programming Examples - Keysight <https://helpfiles.keysight.com/csg/n5186/Content/GPSS/SCPI%20Examples.htm>
189. Hardware implementation of FPGA-based spiking attention ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC12453718/>
190. Spiking FPGA Accelerator for Temporal Task-driven ... <https://arxiv.org/html/2507.10561v1>
191. Complete Tesla Voice Commands List 2026 (Model 3/Y/S ... https://www.basenor.com/blogs/news/tesla-voice-commands-complete-list?srsId=AfmBOoriFPoz9XEp64h2in_EjHoGXBtpj-ZrSUKpSK6siARcrcF1l_0m
192. How to control your Tesla-vehicle with voice commands <https://www.shop4tesla.com/en/pages/tesla-ratgeber-sprachsteuerung>
193. MAVLink Introduction | Middleware for Open Source Drones - YouTube <https://www.youtube.com/watch?v=iZ-usX1VXRI>
194. TrIdent - An R package to automate transductomics analysis of virus-like ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC13060214/>
195. MAVLink Message ID Sequence Dataset - HCRL <https://ocslab.hksecurity.net/Datasets/mavlink-message-id-sequence-dataset>
196. MAVLink Message ID Sequence Dataset - IEEE DataPort <https://ieee-dataport.org/documents/mavlink-message-id-sequence-dataset>
197. Figure 1 from Data analysis of the MAVLink communication protocol <https://www.semanticscholar.org/paper/Data-analysis-of-the-MAVLink-communication-protocol-Atoev-Kwon/1885b848b4276595e529d25274ae1d3fa6f63c18/figure/0>
198. Research Publication Map - MIT DeCode lab <https://decode.mit.edu/publications/>
199. Avionics / Software Development / MAVLink / MAVLink Decoder - GitLab <https://git.skywarder.eu/avn/swd/mavlink/mavdecoder>
200. Free Low-Cost HMI Software for Modbus/TCP PLC Communication <https://industrialmonitordirect.com/blogs/knowledgebase/free-low-cost-hmi-software-for-modbustcp-plc-communication?srsId=AfmBOorXLETpmuQNww9jSiRHTtKqrT49t9qZOo6EpXL3ruPQIImyU25I>

201. Looking for cheap (free?) modbus HMI for Windows : r/PLC - Reddit https://www.reddit.com/r/PLC/comments/13fh8l4/looking_for_cheap_free_modbus_hmi_for_windows/
202. Modbus RTU and TCP Products from ICP DAS USA - YouTube <https://www.youtube.com/watch?v=ejosa4quYbQ>
203. <https://www.se.com/us/en/product-subcategory/5140-...> <https://www.se.com/us/en/product-subcategory/5140-pac-plc-programming-software/undefined/product-comparator/wqbgp//>
204. Modbus SCADA HMI Systems | ICP DAS USA Inc - Data Acquisition https://www.icpdas-usa.com/modbus-scada-hmi-systems.html?srsId=AfmBOoov6wgzy8ITRd1Okqnbq7Bsc4XBZj-9CpXuYT3DJe6DHlpV_Q-5
205. HMI Software | Graybar Store <https://www.graybar.com/human-machine-interface-hmi/c/hmi-software?enablePartNumberSearch=false>
206. <https://www.plm.automation.siemens.com/global/es/p...> <https://www.plm.automation.siemens.com/global/es/products/embedded-software/?cmpid=12818>
207. The Ultimate Terminal Stack in 2026: A Cross-Platform ... <https://medium.com/vmacwrites/the-ultimate-terminal-stack-in-2026-a-cross-platform-guide-for-macos-linux-and-windows-c0d1f93cd9cc>
208. Best Open Source Terminal Emulators 2026 <https://sourceforge.net/directory/terminal-emulators/>
209. fish-shell/fish-shell: The user-friendly command line shell. - GitHub <https://github.com/fish-shell/fish-shell>
210. PowerShell | Red Canary Threat Detection Report <https://redcanary.com/threat-detection-report/techniques/powershell/>
211. IEC 61131-3: PLC Languages & Edition 4 Guide <https://plcprogramming.io/pillar/iec-61131-3-standards>
212. Overview of IEC 61131-3 Programming | PDF - Scribd <https://www.scribd.com/document/384661654/intro-iec-march2013-pdf>
213. IEC 61131-3 Structured Text: A Practical Guide for Modern PLC ... <https://voltrus.id/voltrus-plc/blog/iec-61131-3-structured-text-guide/>
214. IEC 61131-3:2025 Edition 4 — IL Removed, OOP Added <https://t-ia-connect.com/en/iec-61131-3-2025>
215. International standard IEC 61131-3:2013 - Afnor EDITIONS <https://www.boutique.afnor.org/en-gb/standard/iec-6113132013/programmable-controllers-part-3-programming-languages/xs125233/245595>
216. IEC 61131-3 - Lenze <https://www.lenze.com/en-de/application-knowledge-base/article/280455/1>

217. iec-61131-3 · GitHub Topics <https://github.com/topics/iec-61131-3>
218. SIST EN 61131-3:2013 - PLC Programming Languages Standard ... https://standards.iteh.ai/catalog/standards/sist/0d457712-9a99-4d1b-829e-19d8af3b8052/sist-en-61131-3-2013?srsltid=AfmBOopMDyKCWU5kvPIEGh7rs6oV_5sDL8La88r2Ku7M9uXQ1XjAEJcS
219. [PDF] An Open Source IEC 61131-3 Integrated Development Environment <https://beremiz.org/collect/beremiz-paper-indin07.pdf>
220. Dialplan with DTMF <https://community.asterisk.org/t/dialplan-with-dtmf/70380>
221. Chapter 14: Dialplan Application Reference | FreeSWITCH ... <https://developer.signalwire.com/freeswitch/dialplan/dptools>
222. getting a bash returned value into asterisk dialplan <https://stackoverflow.com/questions/14143481/getting-a-bash-returned-value-into-asterisk-dialplan>
223. The Asterisk PBX as a Linked Repeater Controller https://www.qsl.net/kb9mwr/projects/voip/app_rpt_article.pdf
224. Asterisk — Dial plans and related terminologies <https://medium.com/@aqeelabbas3972/asterisk-dial-plans-and-related-terminologies-d4bf724b26c8>
225. 3GPP TS 22.090 V11.0.0 (2012-09) https://www.arib.or.jp/english/html/overview/doc/STD-T63V12_10/5_Appendix/Rel11/22/22090-b00.pdf
226. 3GPP TS 22.090 | PDF | 3 Gpp | Computer Network <https://www.scribd.com/doc/166276783/22090-600>
227. AT Commands <https://radekp.github.io/qtmoko/api/atcommands.html>
228. Specification # 24.090 <https://www.3gpp.org/dynareport/24090.htm>
229. PLC Communication Protocols Documentation Reference ... <https://industrialmonitordirect.com/blogs/knowledgebase/plc-communication-protocols-documentation-reference-guide?srsltid=AfmBOooiQgezUmW7pdbusPpT2S9Q0mZ8UGTmLzbvRwXK4QNkgOLYzlj>
230. Hands-On ROS2 - Part 2 (Service/Client) <https://www.youtube.com/watch?v=RJFoM-vnDJo>
231. Build Your Own PLC Firmware: Step-by-Step Guide with ... <https://www.youtube.com/watch?v=RC9uxR9hDV0>
232. Set up Mavlink connection and serialize messages <https://in.mathworks.com/matlabcentral/answers/2178393-set-up-mavlink-connection-and-serialize-messages>
233. Toward Full-Stack In Silico Synthetic Biology: Integrating ... <https://pubs.acs.org/asbcd6/article/10/8/1931/494828/Toward-Full-Stack-In-Silico-Synthetic-Biology>
234. Support voice actions | Android for Cars <https://developer.android.com/training/cars/media/voice-actions>

235. How To Get CarPlay or Android Auto on Your Tesla <https://www.youtube.com/watch?v=5jIdw-LOXt0>
236. Tesla takes on Android Auto and CarPlay, opens its cars to ... https://www.arenaev.com/tesla_releases_an_api_for_supporting_thirdparty_apps_in_its_cars_similarly_to_carplay_and_android_aud-news-2551.php
237. App Shortcuts | Apple Developer Documentation <https://developer.apple.com/design/human-interface-guidelines/app-shortcuts>
238. Introducing the SmartThings Edge Device Builder with Step ... <https://developer.samsung.com/smartthings/blog/en/2024/02/05/introducing-the-smartthings-edge-device-builder-with-step-by-step-tutorial>
239. obmaz/smartthings_edge_driver: Smartthings Edge Driver https://github.com/obmaz/smartthings_edge_driver
240. Shell vs CLI vs BASH vs TERMINAL (Last but not least Command prompt) https://www.reddit.com/r/learnprogramming/comments/1sy8r6u/shell_vs_cli_vs_bash_vs_terminal_last_but_not/
241. State of Linux Terminal Emulators in 2026 - DEV Community <https://dev.to/shrsrv/state-of-linux-terminal-emulators-in-2026-1gh5>
242. Basics 07: Connecting OpenPLC to an HMI (ScadaBR) <https://www.youtube.com/watch?v=KrcL6lhAHKw>
243. Best Free HMI Software for Modbus/TCP | Comparison Guide <https://industrialmonitordirect.com/blogs/knowledgebase/free-low-cost-hmi-software-for-modbustcp-plc-communication?srsltid=ARcRdnr6Gq1pmn5f7iz8Mo0IUEuGqi4XQwJVek9wmqxz5Q5a0jnUlmTb>
244. Modbus SCADA HMI Systems Training | January 2024 <https://www.youtube.com/watch?v=MNmbG1U59Fw>
245. Engineer's Guide to Modbus Testing, Simulation & ... <https://www.icpdas-usa.com/engineers-guide-to-modbus-testing-simulation-and-programming.html?srsltid=ARcRdnqgkNh4HcWFyvw4f8TG2XQ9Xfoao87ltgnWJIFYpVl5k1nqzNGv>
246. Video Tutorial: Real-Time Collection for Modbus Devices <https://softwaretoolbox.com/blog/video-tutorial-modbus-ethernet-topserver>
247. HMI Programming Tutorial: Build a PLC Operator Screen <https://plcprogramming.io/blog/hmi-programming-tutorial-complete-guide>
248. HMI Hardware and Linux Integration <https://openplc.discussion.community/post/hmi-hardware-and-linux-integration-11569724>
249. Modbus Protocol <https://www.modbustools.com/modbus.html>

- 250. Modbus RTU made simple with detailed descriptions and ... <https://ipc2u.com/articles/knowledge-base/modbus-rtu-made-simple-with-detailed-descriptions-and-examples/>
- 251. Modbus Protocol Overview with Examples <https://www.fernhillsoftware.com/help/drivers/modbus/modbus-protocol.html>
- 252. The MODBUS Serial Communication Protocol <https://community.element14.com/learn/learning-center/the-tech-connection/w/documents/4114/the-modbus-serial-communication-protocol>
- 253. advanced Modbus RTU communication guide <https://www.virtual-serial-port.org/articles/modbus-rtu-guide/>
- 254. Modbus Protocol Guide <https://maplesystems.com/modbus-protocol/?srsltid=AfmBOoqwsff-Tl2gG6vIjpOGutuAC0q0CbwILx2aQwWe32uGp2oLZSW3>
- 255. Modbus RTU Tutorial: RS-485 Wiring, Frames & CRC <https://plcprogramming.io/blog/modbus-rtu-protocol-tutorial-complete-guide>
- 256. A Beginner's Guide to Industrial Communication Protocol <https://modbuskit.com/en/blog/modbus-rtu-basic-tutorial>
- 257. Extracting Firmware from Linux Router using the U-Boot ... <https://www.youtube.com/watch?v=006ROXEYSeI>
- 258. How to continue with interrupted u-boot process via UART ... https://www.reddit.com/r/hardwarehacking/comments/marwq1/how_to_continue_with_interrupted_uboot_process/
- 259. I hack, U-Boot <https://synacktiv.com/publications/i-hack-u-boot>
- 260. Analysis of BusyBox Shell router U-Boot firmware <https://reverseengineering.stackexchange.com/questions/22776/analysis-of-busybox-shell-router-u-boot-firmware>
- 261. Booting armbian manually from u-boot shell over UART <https://forum.armbian.com/topic/57339-booting-armbian-manually-from-u-boot-shell-over-uart/>
- 262. U-Boot overview - stm32mpu - ST wiki https://wiki.st.com/stm32mpu/wiki/U-Boot_overview
- 263. Getting started with U-Boot <https://krinkinmu.github.io/2023/08/12/getting-started-with-u-boot.html>
- 264. Chapter 48: Embedded Scripting APIs | FreeSWITCH ... <https://developer.signalwire.com/freeswitch/programming/scripting-apis>
- 265. FreeSWITCH Modules for Asterisk Developers | PPT <https://www.slideshare.net/slideshow/freeswitch-modules-for-asterisk-developers/15341280>
- 266. ITU-T Recommendation Q.24 https://www.itu.int/rec/dologin_pub.asp?lang=e&id=T-REC-Q.24-198811-I!!PDF-E&type=items

267. What are IEC 61131-3 and PLCopen? - Design World <https://www.designworldonline.com/what-are-iec-61131-3-and-plcopen/>
268. Structured Text Programming: IEC 61131-3 PLC Platform Guide https://industrialmonitordirect.com/blogs/knowledgebase/structured-text-programming-iec-61131-3-platform-guide?srsltid=AfmBOoqACvVW0smHfAAdxDdaC9Z2UK3QgL2qWzc5nzBQ9y_b99pxpPpr
269. What's new in Shortcuts for iOS, iPadOS, macOS, watchOS ... <https://support.apple.com/en-gb/125148>
270. Vehicle Endpoints | Tesla Fleet API <https://developer.tesla.com/docs/fleet-api/endpoints/vehicle-endpoints>
271. Announcements | Tesla Fleet API <https://developer.tesla.com/docs/fleet-api/announcements>
272. Matter Hub simple setup to link Google Home, Alexa and ... <https://community.home-assistant.io/t/matter-hub-simple-setup-to-link-google-home-alexa-and-siri-to-home-assistant/792902>
273. Set up, manage, and control Matter-enabled devices with ... <https://support.google.com/googlehome/answer/13127223>
274. Control a Matter device <https://developers.home.google.com/matter/integration/control>
275. 8- Developing an HMI program for Siemens PLC using Python ... <https://www.youtube.com/watch?v=fOIZ3FV0rgM>
276. Basics 06: Communicate with OpenPLC Runtime using Python <https://www.youtube.com/watch?v=c4esqcNun7k>
277. Low-Cost Free HMI Options for Modbus Serial Applications https://industrialmonitordirect.com/blogs/knowledgebase/low-cost-free-hmi-options-for-modbus-serial-applications?srsltid=AfmBOor8odxJf0eeC4mFQxTBESJzLJVuX0LBldPC-J5vcM_BYs1gekgk
278. Training Videos - Modbus Data Acquisition Controllers <https://www.icpdas-usa.com/modbusdaqtrainingvideo.html?srsltid=AfmBOopWTSrfMnecmDgBFImQq7LWhvNV8FmXdQb1kJawIICrT8PUuevO>
279. 16- Voice Interaction with a PLC and HMI Using Python <https://www.youtube.com/watch?v=0TI1-E6b2IO>
280. Sequential PLC Programming and HMI Development ... <https://www.youtube.com/watch?v=NSKGNykgDmw>
281. How to Connect an HMI+PLC with a Remote I/O via Modbus <https://www.youtube.com/watch?v=qHYuwHfaCM8>
282. Best .NET MODBUS Libraries for Industrial Automation ... <https://industrialmonitordirect.com/blogs/knowledgebase/finding-native-net-modbus->

libraries-for-industrial-applications?

rsrsltid=AfmBOoq3_IW_eevM3veOINSy3i0E7n5Ne3jhmRJGdwQ2NfaVg0xrbYGG

- 283. What's the difference between terminal, shell, command ... https://www.reddit.com/r/learnprogramming/comments/6xr0l9/whats_the_difference_between_terminal_shell/
- 284. What is the Difference Between the Shell, Terminal, and ... https://www.youtube.com/watch?v=Z6_ja-c2pLc
- 285. Understanding Terminal, CLI, Shell, and Operating System ... <https://medium.com/@azama/understanding-terminal-cli-shell-and-operating-system-os-ab910b1e6eb0>
- 286. Comparison of command shells https://en.wikipedia.org/wiki/Comparison_of_command_shells
- 287. What is the difference between Terminal/Shell? https://www.reddit.com/r/commandline/comments/10r4squ/what_is_the_difference_between_terminalshell/
- 288. A Closer Look at the Terminal, Shell, Bash and ... <https://medium.com/meetcyber/a-closer-look-at-the-terminal-shell-bash-and-the-command-line-7b5b858d8790>
- 289. Terminal, Shell, Bash, Command Line and everything in ... <https://www.linkedin.com/pulse/terminal-shell-bash-command-line-everything-between-gal-toubul>
- 290. Recent Publications <https://genometdcc.org/publications/>
- 291. Publications <https://www.futuregenomics.tech/publications/>
- 292. (PDF) Device-to-Device Communications Underlying UAV- ... https://www.researchgate.net/publication/325918140_Device-to-Device_Communications_Underlying_UAV-Supported_Social_Networking
- 293. Selected publications - Khanh Lee's Page <https://www.aibiomedlab.com/publications/>
- 294. Genome <https://www.researchgate.net/journal/Genome-0831-2796>
- 295. UA Cloudflare <https://ir-api.ua.edu/api/core/bitstreams/18e0e4f7-590b-416d-adfe-81b77655dfba/content>
- 296. Publications - UN Laboratory <https://unlab.tech/publications/>
- 297. Publications <https://hmshan.io/publications/>
- 298. (PDF) Experimental Evaluation of Acoustic Drone Tracking ... https://www.researchgate.net/publication/383877168_Experimental_Evaluation_of_Acoustic_Drone_Tracking_using_Mobile_Microphone_Arrays
- 299. (PDF) Analysis and Testing of Data Link Time of Flight ... https://www.researchgate.net/publication/377802551_Analysis_and_Testing_of_Data_Link_Time_of_Flight_measurements_in_Data_Fusion_Algorithms_for_Positioning_of_UAV

300. mavros_msgs/CommandCode Message http://docs.ros.org/en/melodic/api/mavros_msgs/html/msg/CommandCode.html
301. How to write Karel programs? <https://www.robot-forum.com/robotforum/thread/15289-how-to-write-karel-programs/>
302. FANUC KAREL Programming: An Introduction (with ... <https://www.onerobotics.com/posts/2013/introduction-to-karel-programming/>
303. IEC 61131-3 PLC Selection Guide: Standards vs. Practical... <https://industrialmonitordirect.com/blogs/knowledgebase/iec-61131-3-plc-selection-guide-standards-vs-practical-implementation?srsId=AfmBOopNFwdJBOMTJq3nNPi8gyKZyTDOzcMjO41SB-CDu5VNrleyILEk>
304. Introduction to IEC 61131-3: Master the PLC Programming ... <https://www.youtube.com/watch?v=N0Se8Rj96cU>
305. IEC 61131-3:2013 <https://webstore.iec.ch/en/publication/4552>
306. AGI https://docs.asterisk.org/Latest_API/API_Documentation/Dialplan_Applications/AGI/
307. A Technical Introduction to the Asterisk Gateway Interface (AGI) <https://rdegges.com/2010/a-technical-introduction-to-the-asterisk-gateway-interface-agi/>
308. Asterisk Gateway Interface <https://www.youtube.com/watch?v=8lM8AaTdJPU>
309. A Primer to AGI: Asterisk Gateway Interface <https://www.linux.com/news/primer-agi-asterisk-gateway-interface/>
310. Asterisk cmd AGI <https://www.voip-info.org/asterisk-cmd-agi/>
311. Asterisk: The Future of Telephony, 2nd Edition <https://www.oreilly.com/library/view/asterisk-the-future/9780596510480/ch09.html>
312. Resurrecting a Silent Console: From U-Boot to Linux shell <https://www.nozominetworks.com/blog/resurrecting-a-silent-console-from-u-boot-to-linux-shell>
313. How to send binary flashing file to embedded system with ... <https://stackoverflow.com/questions/44595089/how-to-send-binary-flashing-file-to-embedded-system-with-only-serial-console>
314. Configuring Serial Port Debug Console (Linux/U-Boot) <https://developer.toradex.com/software/development-resources/configuring-serial-port-debug-console-linux-u-boot>
315. Configure uboot to leave serial console ttyS0 to Linux <https://forum.doozan.com/read.php?3,1565635871,newer>
316. How to stop booting in U-Boot and get U-Boot command ... <https://forum.radxa.com/t/how-to-stop-booting-in-u-boot-and-get-u-boot-command-line/5248>

317. Firmware Acquisition: U-Boot - Embedded Lab Vienna for IoT ... https://wiki.elvis.science/index.php?title=Firmware_Acquisition:_U-Boot
318. Tutorial — fish-shell 4.8.1 documentation <https://fishshell.com/docs/current/tutorial.html>
319. Set up and use Google Assistant or Amazon Alexa with ... <https://www.samsung.com/us/support/answer/ANS10006871/>
320. About voice interaction https://source.android.com/docs/automotive/voice/voice_interaction_guide
321. A localized DNA finite-state machine with temporal resolution <https://www.science.org/doi/10.1126/sciadv.abm9530>
322. A last-in first-out stack data structure implemented in DNA <https://www.nature.com/articles/s41467-021-25023-6>
323. Synthetic recombinase-based state machines in living cells <https://www.science.org/doi/abs/10.1126/science.aad8559>
324. A spatially localized DNA linear classifier for cancer diagnosis <https://pmc.ncbi.nlm.nih.gov/articles/PMC11136972/>
325. Convolutional Differentiable Logic Gate Networks <https://arxiv.org/html/2411.04732v1>
326. IEEE 488.2 Commands https://dl.cdn-anritsu.com/en-us/test-measurement/ohs/10450-00040AE/Programming_Commands/PM_Ch2_ProgramWithSCPI.03.03.html
327. IEEE-488.2 (GPIB) Software Configuration Application Note https://www.mpgdover.com/content/dam/mpg/documents/manuals/CANbus_switch_GPIB_configuration-DKM.pdf
328. Oscilloscopes Remote Control and Automation Manual <https://www.e-sonic.com/Upload/LeaderBoard/175cd5ce-3151-44e4-8342-9290b11c9944.pdf?srsltid=AfmBOoo4LOywiVQkrwyVKaFS5llBhpZ6U14IfDAFRf3jYMDas-YCTtDr>
329. User's Guide Models 612 and 614 Cryogenic Temperature ... https://cryomagnetics.com/wp-content/uploads/2022/05/Model_612_and_614_Operations_Manual-R2.pdf
330. Sending URScript code to the robot https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/doc/ur_robot_driver/ur_robot_driver/doc/usage/script_code.html
331. Sending urscript in remote control mode while UR_Driver ... https://github.com/UniversalRobots/Universal_Robots_ROS2_Driver/issues/982
332. Guide and example: UR Remote Control via TCP/IP <https://forum.universal-robots.com/t/guide-and-example-ur-remote-control-via-tcpip/32973>

333. A practitioner's guide to ros2_control | Videos & Movies on ... <https://vimeo.com/767139648>
334. [MAVLink] Introduction to MAVLink, Ardupilot and its Hardware ... <https://www.youtube.com/watch?v=qLfxzeKu2Hg>
335. Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey <https://arxiv.org/pdf/1906.10641>
336. Data analysis of the MAVLink communication protocol https://www.researchgate.net/publication/321816339_Data_analysis_of_the_MAVLink_communication_protocol
337. Engineering Degree Project Streamlining UAV ... <https://www.diva-portal.org/smash/get/diva2:1781009/FULLTEXT01.pdf>
338. AGU25 <https://agu.confex.com/agu/agu25/meetingapp.cgi/Paper/1882749>
339. <https://data.ess-dive.lbl.gov/view/doi:10.15485/16...> <https://data.ess-dive.lbl.gov/view/doi:10.15485/1644687>
340. Deep Learning on Multi Sensor Data for Counter UAV ... - PMC <https://pmc.ncbi.nlm.nih.gov/articles/PMC6891421/>
341. Recommendations for Low-Noise Data Acquisition with ... <https://www.mdpi.com/2072-4292/18/15/2564>
342. A Survey on the Use of Deep Learning Techniques for UAV ... <https://www.mdpi.com/2079-9292/11/19/3025>
343. Vuk Marojevic https://www.gnuradio.org/grcon/grcon19/presentations/UAS_Community_Testbed_Architecture/Vuk%20Marojevic%20-%20UAS%20Community%20Testbed.pdf
344. sirredbeard/awesome-unix: All the UNIX and UNIX-Like: ... <https://github.com/sirredbeard/awesome-unix>
345. What is the best operating system with a command line ... <https://www.quora.com/What-is-the-best-operating-system-with-a-command-line-interface-Android-or-Linux-Unix-based>
346. Knowing that macOS is Unix-based and that Linux is a ... <https://www.quora.com/Knowing-that-macOS-is-Unix-based-and-that-Linux-is-a-Unix-like-operating-system-how-closely-related-are-Linux-and-the-Apple-operating-system>
347. Comparison of real-time operating systems https://en.wikipedia.org/wiki/Comparison_of_real-time_operating_systems
348. What UNIX-based operating systems have support for GUI ... <https://www.quora.com/What-UNIX-based-operating-systems-have-support-for-GUI-that-is-at-least-as-strong-as-macOSs>
349. What's the difference between a RTOS and a regular OS? https://www.reddit.com/r/embedded/comments/ybq5g9/whats_the_difference_between_a_rtos_and_a_regular/

350. What are the five most commonly used real-time operating ... <https://stackoverflow.com/questions/5281848/what-are-the-five-most-commonly-used-real-time-operating-systems>
351. Different Operating Systems <https://www.geeksforgeeks.org/operating-systems/different-operating-systems/>
352. Top 10 Unix-Based Operating Systems for 2026 (Expert ... <https://www.fosslinux.com/44623/top-unix-based-operating-systems.htm>
353. Low-Cost Free HMI Options for Modbus Serial Applications <https://industrialmonitordirect.com/blogs/knowledgebase/low-cost-free-hmi-options-for-modbus-serial-applications?srsId=AfmBOoqQ13fpdS6VaZSr8dhVDZ2Yk48QPyKXQRsB8OXCZ66Vh7F-1QgT>
354. Training Videos - Modbus Data Acquisition Controllers https://www.icpdas-usa.com/modbusdaqtrainingvideo.html?srsId=AfmBOooJ4lMzJCn8Bslv8ac4Nk95o02mQd_A0ksYqvMyEoWOVfZPlmE9
355. PLC HMI Tutorial | FactoryTalk View & Easy Builder 8000 ... <https://www.youtube.com/watch?v=HR84q631dLM>
356. FRP Technology: Custom Access Control Integration <https://controlbyweb.com/blog/custom-access-control-integration/?srsId=AfmBOor9qUSffPFOBdlGtQCnStD3NOyGP3c0DCLBJSCOocNTbFMa8n4l>
357. Voicebots and CRMs FreeSWITCH AI Integration via ESL <https://www.ecosmob.com/blog/voicebots-crms-freeswitch-ai-integration-esl/>
358. Embedded Systems at IFA 2025 <https://logicmachine.net/ifa-2025/>
359. Why Edge Intelligence Is the Future of Industrial Control <https://www.digi.com/blog/post/edge-intelligence-future-of-industrial-control>
360. System-level FPGA validation of a trainable and robust ... <https://www.sciencedirect.com/science/article/pii/S258900422601360X>
361. ModNEF : An Open Source Modular Neuromorphic ... <https://dl.acm.org/doi/full/10.1145/3730581>
362. SCPI for Embedded Systems <https://www.maskset.net/blog/2025/08/26/scpi-for-embedded-systems/>
363. shortcutItems | Apple Developer Documentation <https://developer.apple.com/documentation/uikit/uiapplication/shortcutitems>
364. Computer-Aided Whole-Cell Design: Taking a Holistic ... <https://www.frontiersin.org/journals/bioengineering-and-biotechnology/articles/10.3389/fbioe.2020.00942/full>
365. Getting Started with Matter: Your Guide to a Future-Proof Smart Home https://atios.ch/blogs/news/getting-started-with-matter-your-guide-to-a-future-proof-smart-home?srsId=AfmBOooenw_VOC6yJBRBUZXEBMF-OyWX9JVic-3OJ_G0hMZ-S-A2OcxV

366. SCPI Syntax and Usage https://www.ni.com/docs/en-US/bundle/wireless-test-system-api-overview/page/scpi-usage.html?srsltid=AfmBOorSObhpf_sNnvptFhh6sgMXqSMQs7Bev3BfTDuC7jIebjJlDHk4
367. GPIB Overview - MATLAB & Simulink <https://www.mathworks.com/help/instrument/gpib-overview.html>
368. Programming with SCPI https://dl.cdn-anritsu.com/en-us/test-measurement/ohs/10450-00078C/MS2085A-89A_Site_Master_Help/scpi-remote-programming.63.1.html
369. AGI | MikoPBX development <https://docs.mikopbx.com/mikopbx-development/internal-structure/api/agi>
370. AGI Commands Reference: Control Asterisk Calls from Scripts <https://asteriskrecipes.com/reference/agi-commands/>
371. Research: Neuromorphic Computing and AI Chip Design <https://bricc.ece.vt.edu/Research-1.html>
372. Nanopore decoding with speed and versatility for data storage <https://pmc.ncbi.nlm.nih.gov/articles/PMC11755093/>
373. Sassy: fuzzy searching DNA sequences using SIMD <https://pubmed.ncbi.nlm.nih.gov/42178392/>
374. DMAV: Enhanced MAV Link Protocol Using Dynamic DNA ... https://www.researchgate.net/publication/363156101_DMAV_Enhanced_MAV_Link_Protocol_Using_Dynamic_DNA_Coding_for_Unmanned_Aerial_Vehicles
375. A Simple Example for Decoding DTMF Signals with CUDA ... <https://github.com/MarkusThill/DTMFCuda>
376. DNA as a Double-Coding Device for Information Conversion ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC7617498/>
377. Direct transposition of native DNA for sensitive multimodal ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC11176058/>
378. Dtmf Encoder And Decoder Using Labview <https://peer.asee.org/dtmf-encoder-and-decoder-using-labview.pdf>
379. From Unified Data Collection to Automated Code Generation <https://arxiv.org/pdf/2507.10321>
380. Has anyone moved to an os that supports only the ... https://www.reddit.com/r/commandline/comments/hzg4rk/has_anyone_moved_to_an_os_that_supports_only_the/
381. Can a command line interface be the same for all ... <https://www.quora.com/Can-a-command-line-interface-be-the-same-for-all-the-operating-systems-If-no-then-why>
382. Command-line interface https://en.wikipedia.org/wiki/Command-line_interface

383. Introduction to Command Line Usage in Windows and Linux <https://www.youtube.com/watch?v=XAWUXijFL5Y>
384. What is an example of an operating system that uses ... <https://www.quora.com/What-is-an-example-of-an-operating-system-that-uses-a-command-line-interface>
385. User Operating System Interface || CLI || Command Line ... <https://www.youtube.com/watch?v=dcZXx2n7Xbs>
386. Which OS is known for command-line interface? <https://www.facebook.com/groups/1101826804720582/posts/1255470272689567/>
387. What Is Command Line Interface (CLI)? How to Use It <https://www.g2.com/articles/command-line-interface>
388. The PLC Programming Languages for Controls and ... <https://www.youtube.com/watch?v=gTb-TAT7W4o>
389. Most beneficial software programming language? : r/PLC https://www.reddit.com/r/PLC/comments/171jpu/most_beneficial_software_programming_language/
390. Industrial Automation Programming Languages <https://industrialmonitordirect.com/blogs/knowledgebase/industrial-automation-programming-languages-a-technical-reference-for-st-c-c-python-and-related-tech?srltid=AfmBOoo1eOy8R30HDVmi916JntB7Wli6Ft8pmw6Z0Hp4CYARIlgvvRre>
391. Scripting Language for DevOps | Best Scripting Language for ... <https://www.youtube.com/watch?v=YzaQcJ5xSTk>
392. Implementing Automation with Scripting Languages ... <https://www.youtube.com/watch?v=NIgqwpZnlbA>
393. Best Programming Languages for Industrial Automation ... https://industrialmonitordirect.com/blogs/knowledgebase/programming-languages-for-industrial-automation-plc-programmers-guide?srltid=AfmBOoqOFqS_QXGOVXJ2Jgs6p5ubSBzR6VJy3oE3Bmb3qEYLHbdVndUQ
394. Industrial Automation Programming: PLC, HMI, SCADA & ... <https://plcprogramming.io/blog/industrial-automation-programming-guide>
395. The Coding Languages of Industrial Automation <https://www.mroelectric.com/blog/the-coding-languages-of-industrial-automation/>
396. 9 Best Programming Languages for Test Automation <https://www.youtube.com/watch?v=0h4sxghBNQs>
397. Help (New to automation) What Programming language ... https://www.reddit.com/r/automation/comments/1aqujgn/help_new_to_automation_what_programming_language/